

Arduino 基础

机器人工程 2302 金加康 整理排版

(强烈建议对机器人感兴趣的 xpy 们先学学 ROS，该教程某些部分需要一定的 ROS 基础)

一、前言

机器人系统是一套机电一体化的设备，机器人设计也是高度集成的系统性实现，为了给大家解答上述疑惑，方便机器人硬件的快速上手，本教程去繁就简旨在从 0 到 1 的设计一款入门级、低成本、简单但又具备一定扩展性的两轮差速机器人，学习完本章内容之后，你甚至可以构建属于自己的机器人平台。

本教程主要介绍内容如下：

- 机器人的组成部分；
- Arduino 基本使用；
- Arduino 与电机驱动；
- 底盘控制实现；
- 基于树莓派的 ROS 环境搭建；
- 激光雷达与相机的基本使用与集成。

本教程学习目标如下：

- 能够独立搭建机器人平台。

注意：

- 该教程内容会使用到 ROS 的分布式框架，树莓派端作为主机，PC 端作为从机；
- PC 端使用的 ROS 版本为 noetic，树莓派端使用的版本为 melodic，因为树莓派需要与底盘交互，而相关功能包还未更新。
- 为保证教程的连续性和完整性，在两轮差速机器人中未用到，但是在 Arduino 使用过程中常用的器件，我会在介绍完两轮差速机器人的完整设计后，在补充章节再作介绍。

【案例演示】

1. 机器人底盘实现

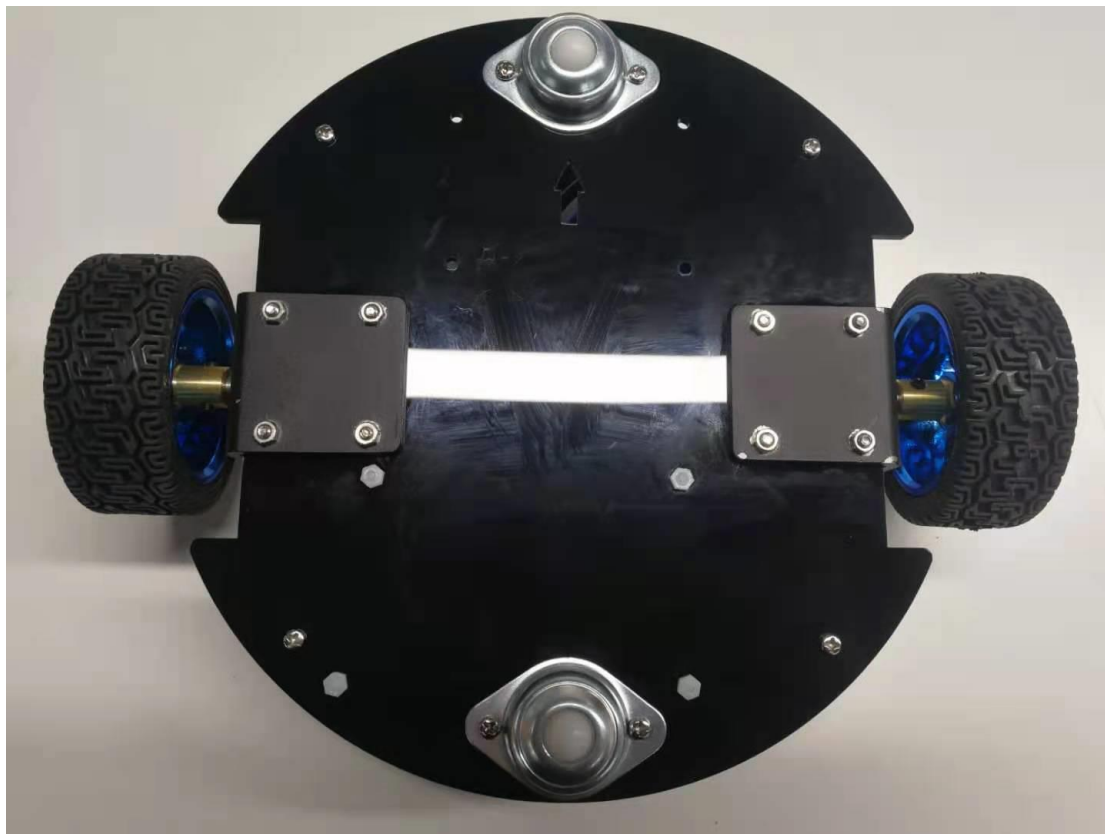
底盘正面



Arduino 与 电机驱动板



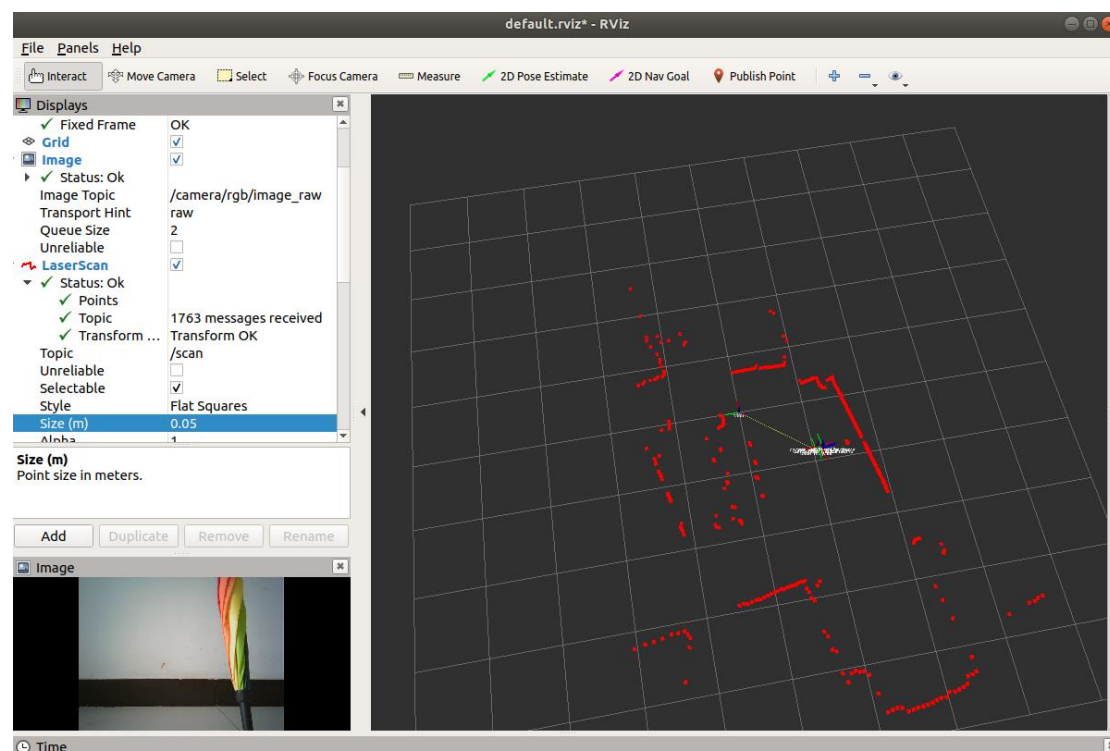
底盘背面



2. 机器人控制系统以及传感器实现



3. 机器人集成效果



【Arduino 部分】

Arduino 是一款便捷灵活、方便上手的开源电子原型平台。在它上面可以进行简单的电路控制设计，Arduino 能够通过各种各样的传感器来感知环境，通过控制灯光、马达和其他的装置来反馈、影响环境。

立足角度不同，对机器人组成的认识也会有明显差异，从控制的角度来看，机器人系统可以分为四部分：

传感系统、控制系统、驱动系统、执行机构。

1. 传感系统

它由内部传感器模块和外部传感器模块组成，获取内部和外部环境中有用的信息，相当于人体的感官与神经，内部传感系统包括电机的编码器、陀螺仪等，可以通过自身信号反馈检测位姿状态；外部传感系统包括摄像头、红外、声纳等，用于感知外部环境。

2. 控制系统

控制系统的任务是根据机器人的作业指令以及从传感器反馈回来的信号，输出控制命令信号，类似于人的大脑。控制系统需要基于处理器实现，在处理器之上，控制系统需要完成算法处理、关节控制、人机交互等复杂功能。

3. 驱动系统

驱动系统主要负责驱动执行机构，将控制系统下达的命令转换成执行机构所需要的信号，相当于人的小脑与神经。采用的动力源不同，驱动系统的传动方式也不同。驱动系统的传动方式主要有四种：液压式、气压式、电气式和机械式。电力驱动是目前使用最多的一种驱动方式，其特点是电源取用方便，响应快，驱动力大，信号检测、传递、处理方便，并可以采用多种灵活的控制方式，驱动电机一般采用步进电机或伺服电机。

4. 执行机构

执行机构是机器人组成中的机械部分，类似于人的手与脚，比如：机器人的行走部分与机械臂。

在当前机器人系统中，各组成部分对应硬件清单如下：

执行机构：主体使用亚克力板拼装，由两个直流电机带动主动轮以及保持平衡的两个万向轮实现机器人行走，由于执行机构比较简单，不再做单独介绍。

驱动系统：电池、arduino 以及电机驱动模块；

控制系统：树莓派；

传感系统：编码器、单线激光雷达、相机；

其中，执行机构与驱动系统构成了机器人底盘。

（大部分材料实验室都有，可以自行组装，也可到淘宝上直接购买整车，但是价格比较贵）

二、Arduino 简介

1. 概念

Arduino 是一款便捷灵活、方便上手的开源电子原型平台。在它上面可以进行简单的电路控制设计，Arduino 能够通过各种各样的传感器来感知环境，通过控制灯光、马达和其他的装置来反馈、影响环境。

2. 作用

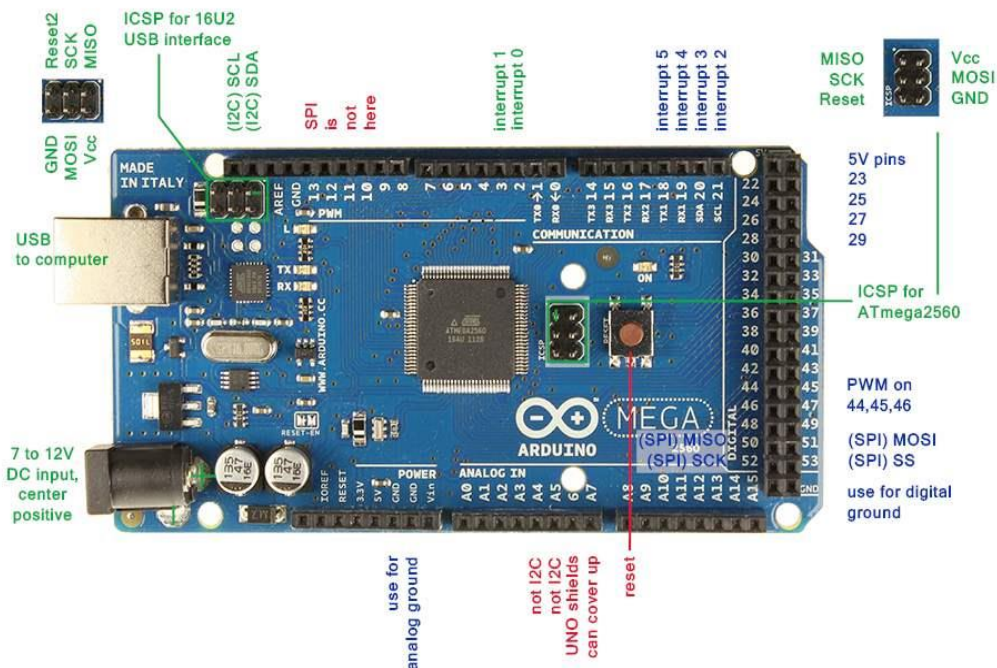
或多或少你可能听说过“集成电路”（又称“微电路”、“微芯片”或“芯片”）这种概念，集成电路（integrated circuit）是一种微型电子器件或部件，通过集成电路再结合一些外围的电子电子元器件、传感器等，可以感知环境（温度、湿度、声音），也可以影响环境（控制灯的开关、调节电机转速）。但是传统的集成电路应用比较繁琐，一般需要具有一定电子知识基础，并懂得如何进行相关的程序设计的工程师才能熟练使用，而 Arduino 的出现才使得以往高度专业的集成电路变得平易近人，Arduino 主要优点如下：

- **简单**: 在硬件方面，Arduino 本身是一款非常容易使用的印刷电路板。电路板上装有专用集成电路，并将集成电路的功能引脚引出方便我们外接使用。同时，电路板还设计有 USB 接口方便与电脑连接；
- **易学**: 只需要掌握 C/C++ 基本语法即可；
- **易用**: Arduino 提供了专门的程序开发环境 Arduino IDE，可以提高程序实现效率。

当前，Arduino 已经成为全世界电子爱好者电子制作过程中的重要选项之一。

3. 组成

Arduino 体系主要包含硬件和软件两大部分。硬件部分是可以用来做电路连接的各种型号的 Arduino 电路板（下图为本章内容使用的 Arduino mega 2560）；软件部分则是 Arduino IDE。你只要在 IDE 中编写程序代码，将程序上传到 Arduino 电路板后，程序便会告诉 Arduino 电路板要做什么了。

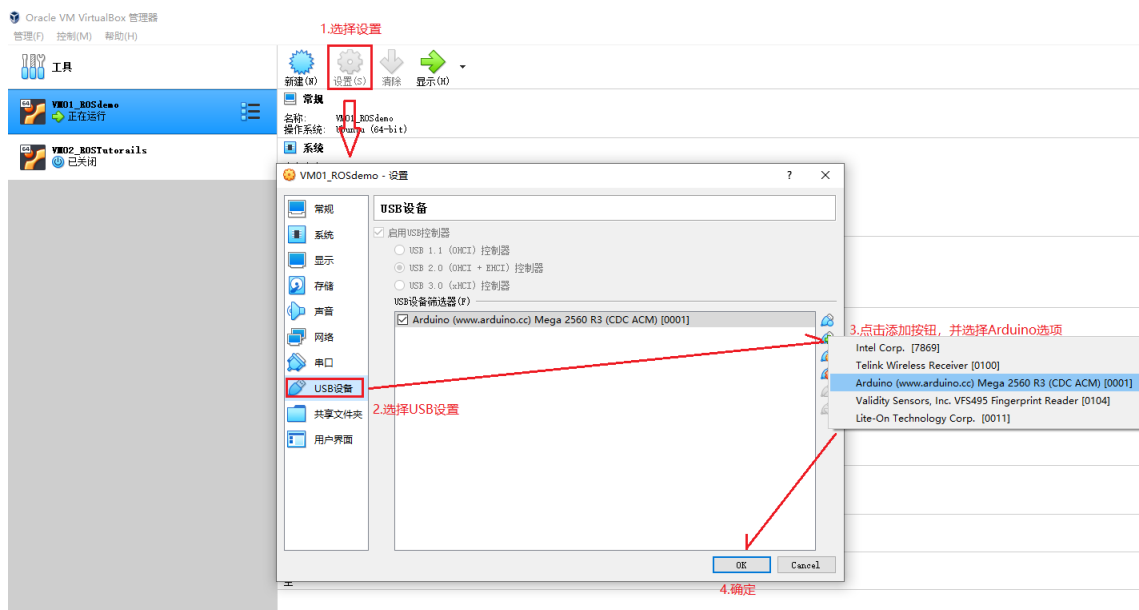


三、Arduino 开发环境搭建（以 Linux 系统为例）

基于 Arduino 的开发实现,毋庸置疑的必须先要准备 Arduino 电路板(建议型号:Arduino Mega 2560),除了硬件之外,还需要准备软件环境,安装 Arduino IDE,在 Ubuntu 下,Arduino 开发环境的搭建步骤如下:

1. 硬件准备: Arduino 电路板连接 ubuntu (Windows 系统这步可省略)
2. 软件准备: 安装 Arduino IDE
3. 编写 Arduino 程序并上传至 Arduino 电路板。

1. Arduino 连接 Ubuntu (Windows 系统这步可省略)



你需要确保你对这个接口有访问的权限。假设你的 Arduino 连接的是 `/dev/ttyACM0`, 那么就运行下面这个命令:

```
ls -l /dev/ttyACM0
```

然后你就可以看到类似于下面的输出结果:

```
crw-rw-- 1 root dialout 166, 0 2013-02-24 08:31 /dev/ttyACM0
```

我们注意到在上面的结果中,只有 root 和 "dialout" 组才有读写权限。因此,你需要成为 dialout 组的一个成员。

命令如下:

```
sudo usermod -a -G dialout your_user_name
```

在这个命令中 `your_user_name` 就是你在 Linux 下登录的用户名。然后需要重启使之生效。执行完上面的操作之后,你可以运行下面的命令查看一下:

```
groups
```

然后如果你可以在列出的组中找到 dialout,这就说明你已经加入到 dialout 中了。

2. 安装 Arduino IDE

2.1. 下载 arduino ide 安装包 (Windows 系统只需这步,后根据提示安装即可)

官方下载链接: <https://www.arduino.cc/en/Main/Software>。

Download the Arduino IDE



ARDUINO 1.8.13

The open-source Arduino Software (IDE) makes it easy to write code and upload it to the board. It runs on Windows, Mac OS X, and Linux. The environment is written in Java and based on Processing and other open-source software. This software can be used with any Arduino board. Refer to the [Getting Started](#) page for Installation instructions.

Windows Installer, for Windows 7 and up
Windows ZIP file for non admin install

Windows app Requires Win 8.1 or 10
[Get](#)

Mac OS X 10.10 or newer

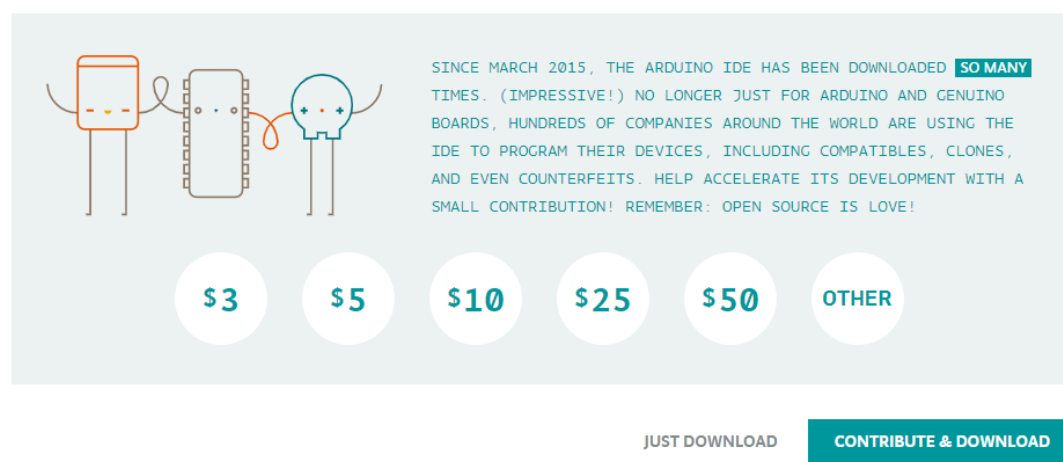
Linux 32 bits
Linux 64 bits
Linux ARM 32 bits
Linux ARM 64 bits

[Release Notes](#)
[Source Code](#)
[Checksums \(sha512\)](#)

选择对应版本即可。

Contribute to the Arduino Software

Consider supporting the Arduino Software by contributing to its development. (US tax payers, please note this contribution is not tax deductible). [Learn more on how your contribution will be used.](#)



SINCE MARCH 2015, THE ARDUINO IDE HAS BEEN DOWNLOADED **SO MANY** TIMES. (IMPRESSIVE!) NO LONGER JUST FOR ARDUINO AND GENUINO BOARDS, HUNDREDS OF COMPANIES AROUND THE WORLD ARE USING THE IDE TO PROGRAM THEIR DEVICES, INCLUDING COMPATIBLES, CLONES, AND EVEN COUNTERFEITS. HELP ACCELERATE ITS DEVELOPMENT WITH A SMALL CONTRIBUTION! REMEMBER: OPEN SOURCE IS LOVE!

\$3 **\$5** **\$10** **\$25** **\$50** **OTHER**

[JUST DOWNLOAD](#) [CONTRIBUTE & DOWNLOAD](#)

可以选择捐款，或者 JUST DOWNLOAD。

2.2. 使用 tar 命令对压缩包解压

```
tar -xvf arduino-1.x.y-linux64.tar.xz
```

2.3. 将解压后的文件移动到/opt 下

```
sudo mv arduino-1.x.y /opt
```

2.4. 进入安装目录, 对 install.sh 添加可执行权限, 并执行安装

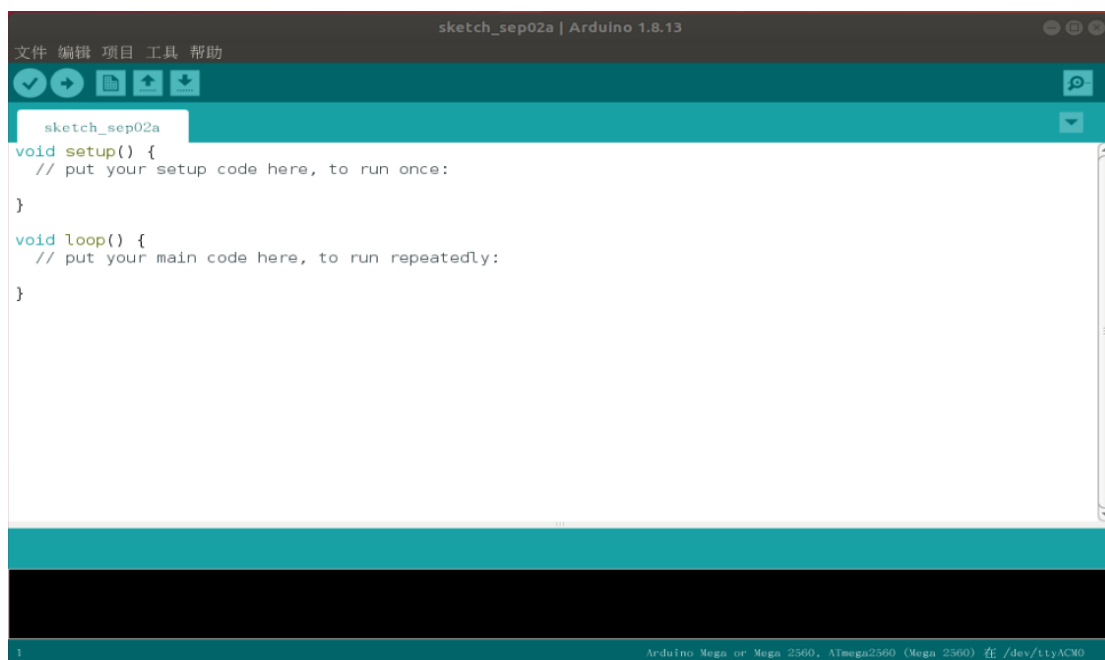
```
cd /opt/arduino-1.x.y
```

```
sudo chmod +x install.sh
```

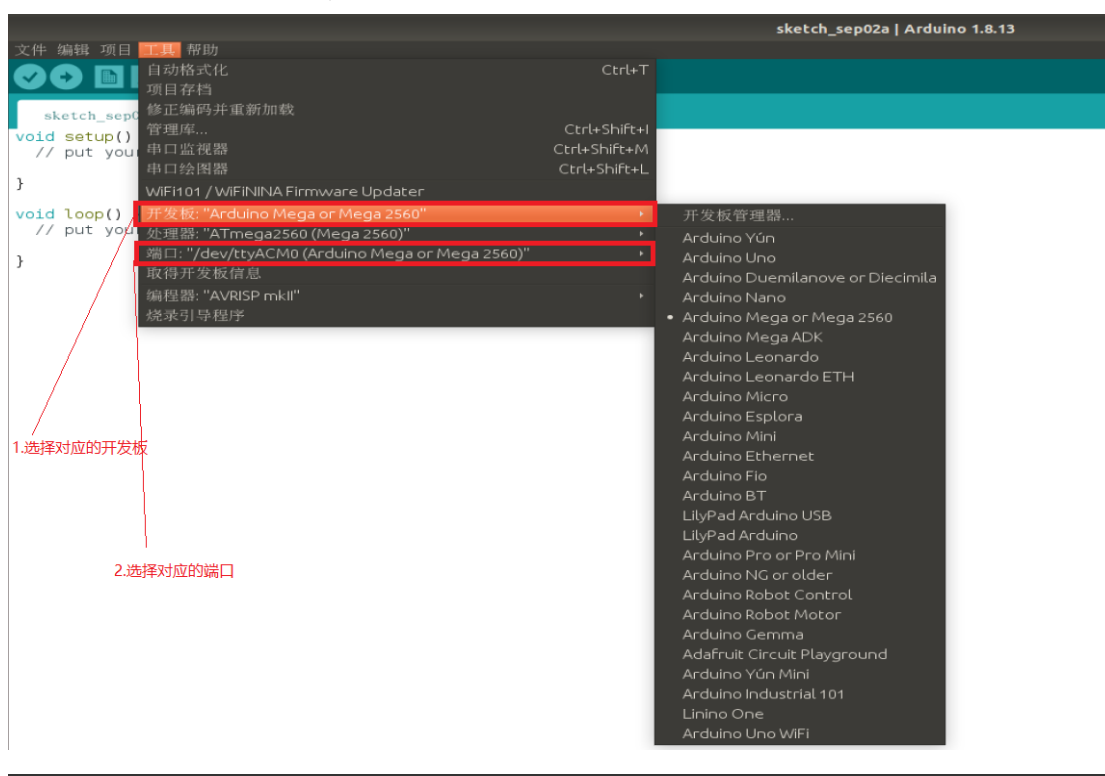
```
sudo ./install.sh
```

2.5. 启动并配置 Arduino IDE

在命令行直接输入:arduino, 或者点击左下的显示应用程序搜索 arduino IDE。启动如下:



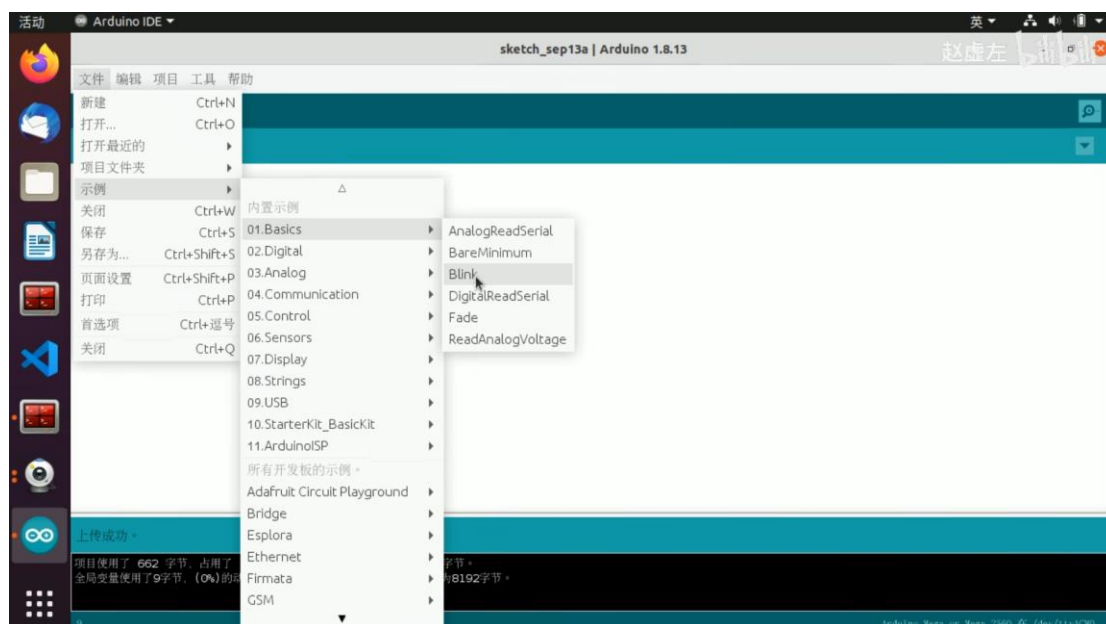
Arduino IDE 配置如下:



3. Hello World 实现

Arduino IDE 中已经内置了一些相关案例, 在此, 我们通过一个经典的控制 LED 等闪烁案例来演示 Arduino 的使用流程:

3.1. 案例调用



3.2. 编译及上传

先点击左上的编译按钮，可用于语法检测，编译无异常，再点击右侧的上传按钮，上传至 Arduino 电路板



3.3. 运行结果

电路板上的 LED 灯闪烁

3.4. 代码解释

// 初始化函数

```
void setup() {
```

```
    //将 LED 灯引脚(引脚值为 13, 被封装为了 LED_BUTLIN)设置为输出模式
```

```
    pinMode(LED_BUILTIN, OUTPUT);
```

```
}
```

// 循环执行函数

```
void loop() {  
    digitalWrite(LED_BUILTIN, HIGH);    // 打开 LED 灯  
    delay(1000);                        // 休眠 1000 毫秒  
    digitalWrite(LED_BUILTIN, LOW);     // 关闭 LED 灯  
    delay(1000);                        // 休眠 1000 毫秒  
}
```

setup 与 loop 函数是固定格式。

四、Arduino 基本语法概述

Arduino 的语言系统在设计时参考了 C、C++、Java，是一种综合性的简洁语言，语法更类似于 C++，但是不支持 C++ 的异常处理，没有 STL 库，你可以把它当作是精简后的 C++。

Arduino 基本语法中，注释、宏定义、库文件包含、变量、函数、流程控制、类、继承、多态..... 都与 C++ 高度类似，在此不再赘述，着重介绍的是，Arduino 中的一些 API 实现。



1. 程序结构

一个 Arduino 程序分为两大部分：setup() 与 loop() 函数。

- **void setup():** 在这个函数里初始化 Arduino 的程序，使主循环程序在开始之前设置好相关参数，初始化变量、设置引脚的输出\输入类型、设置波特率...。该函数只会在上电或重启时执行一次。
- **void loop():** 这是 Arduino 的主函数。这套程序会一直重复执行，直到电源被断开。

2. 常量

在 Arduino 中封装了一些常用常量，比如：

- **HIGH | LOW** (引脚电压定义)
- **INPUT | OUTPUT** (数字引脚 (Digital pins) 定义)
- **true | false** (逻辑层定义)

3. 通信_Serial

Serial 用于 Arduino 控制板和一台计算机或其他设备之间的通信。您可以使用 Arduino IDE 内置的串口监视器与 Arduino 板通信。点击工具栏上的串口监视器按钮，调用 begin() 函数 (选择相同的波特率)。

- **Serial.begin()** 初始化串口波特率

描述：

将串行数据传输速率设置为位/秒 (波特)。与计算机进行通信时，可以使用这些波特率：300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600 或 115200。当然，您也可以指定其他波特率 - 例如，引脚 0 和 1 和一个元件进行通信，它需要一个特定的波特率。

语法: Serial.begin(speed)

参数: speed: 位/秒 (波特) - long

返回: 无

- **Serial.print()** 从串口打印输出数据

需求:

以人们可读的 ASCII 文本形式打印数据到串口输出。此命令可以采取多种形式。每个数字的打印输出使用的是 ASCII 字符。浮点型同样打印输出的是 ASCII 字符, 保留到小数点后两位。Bytes 型则打印输出单个字符。字符和字符串原样打印输出。Serial.print() 打印输出数据不换行, Serial.println() 打印输出数据自动换行处理。

语法: Serial.print(val)

参数

val: 打印输出的值 - 任何数据类型

返回: 字节 print() 将返回写入的字节数, 但是否使用 (或读出) 这个数字是可设定的

- **Serial.println()** 打印输出数据自动换行处理。参考 Serial.print();

- **Serial.available()**

描述:

获取从串口读取有效的字节数 (字符)。这是已经传输到, 并存储在串行接收缓冲区 (能够存储 64 个字节) 的数据。available() 继承了 Stream 类。

语法: Serial.available()

参数: 无

返回: 可读取的字节数

- **Serial.read()**

描述: 读取传入的串口的数据。read() 继承自 Stream 类。

语法: serial.read()

参数: 无

返回: 传入的串口数据的第一个字节 (或 -1, 如果没有可用的数据)

4. 函数_数字 IO

- **pinMode()**

描述: 将指定的引脚配置成输出或输入。

语法: pinMode(pin, mode)

参数

pin: 要设置模式的引脚

mode: INPUT 或 OUTPUT

返回: 无

- **digitalWrite()**

描述: 给一个数字引脚写入 HIGH 或者 LOW。

语法: digitalWrite(pin, value)

参数

pin: 引脚编号 (如 1, 5, 10, A0, A3)

value: HIGH or LOW

返回: 无

- **digitalRead()**

描述: 读取指定引脚的值, HIGH 或 LOW。

语法: `digitalRead (PIN)`

参数

pin: 你想读取的引脚号 (int)

返回: HIGH 或 LOW

注意: 如果引脚悬空, `digitalRead()` 会返回 HIGH 或 LOW (随机变化)

5. 函数_模拟 IO

- **`analogWrite()` PWM**

描述:

从一个引脚输出模拟值 (PWM)。可用于让 LED 以不同的亮度点亮或驱动电机以不同的速度旋转。`analogWrite()` 输出结束后, 该引脚将产生一个稳定的特殊占空比方波, 直到下次调用 `analogWrite()` (或在同一引脚调用 `digitalRead()` 或 `digitalWrite()`)。PWM 信号的频率大约是 490 赫兹。

在大多数 arduino 板 (ATmega168 或 ATmega328), 只有引脚 3, 5, 6, 9, 10 和 11 可以实现该功能。在 arduino Mega 上, 引脚 2 到 13 可以实现该功能。老的 Arduino 板 (ATmega8) 的只有引脚 9、10、11 可以使用 `analogWrite()`。在使用 `analogWrite()` 前, 你不需要调用 `pinMode()` 来设置引脚为输出引脚。

语法: `analogWrite (pin,value)`

参数

pin: 用于输入数值的引脚。

value: 占空比: 0 (完全关闭) 到 255 (完全打开) 之间。

返回: 无

6. 函数_时间

- **`delay()`**

描述: 使程序暂定设定的时间 (单位毫秒)。(一秒等于 1000 毫秒)

语法: `delay(ms)`

参数

ms: 暂停的毫秒数 (unsigned long)

返回: 无

- **`millis()`**

描述:

返回 Arduino 开发板从运行当前程序开始的毫秒数。这个数字将在约 50 天后溢出 (归零)。

参数: 无

返回: 返回从运行当前程序开始的毫秒数 (无符号长整数)。

7. 函数_中断

- **`attachInterrupt()`**

描述:

当发生外部中断时, 调用一个指定函数。当中断发生时, 该函数会取代正在执行的程序。大多数的 Arduino 板有两个外部中断: 0 (数字引脚 2) 和 1 (数字引脚 3)。

arduino Mega 还有其它有四个外部中断: 数字 2 (引脚 21), 3 (引脚 20), 4 (引脚 19), 5 (引脚 18)。

语法: attachInterrupt(interrupt, function, mode)

interrupt: 中断引脚数

function: 中断发生时调用的函数，此函数必须不带参数和不返回任何值。该函数有时被称为中断服务程序。

mode: 定义何时发生中断以下四个 constants 预定有效值:

- LOW 当引脚为低电平时，触发中断
- CHANGE 当引脚电平发生改变时，触发中断
- RISING 当引脚由低电平变为高电平时，触发中断
- FALLING 当引脚由高电平变为低电平时，触发中断。

返回: 无

注意事项:

当中断函数发生时，delay() 和 millis() 的数值将不会继续变化。当中断发生时，串口收到的数据可能会丢失。你应该声明一个变量来在未发生中断时储存数据。

- **noInterrupts() (禁止中断)**

描述:

禁止中断 (重新使能中断 interrupts())。中断允许在后台运行一些重要任务，默认使能中断。禁止中断时部分函数将无法工作，通信中接收到的信息也可能会丢失。中断会稍影响计时代码，在某些特定的代码中也会失效。

参数: 无

返回: 无

- **interrupts() (中断)**

描述:

重新启用中断 (使用 noInterrupts() 命令后将被禁用)。中断允许一些重要任务在后台运行，默认状态是启用的。禁用中断后一些函数可能无法工作，并传入信息可能会被忽略。中断会稍微打乱代码的时间，但是在关键部分可以禁用中断。

参数: 无

返回: 无

Arduino 的 API 还有很多，但是受于篇幅限制，当前只是简单介绍了一些 API 实现。

五、Arduino 基本语法演示

1. 通信操作

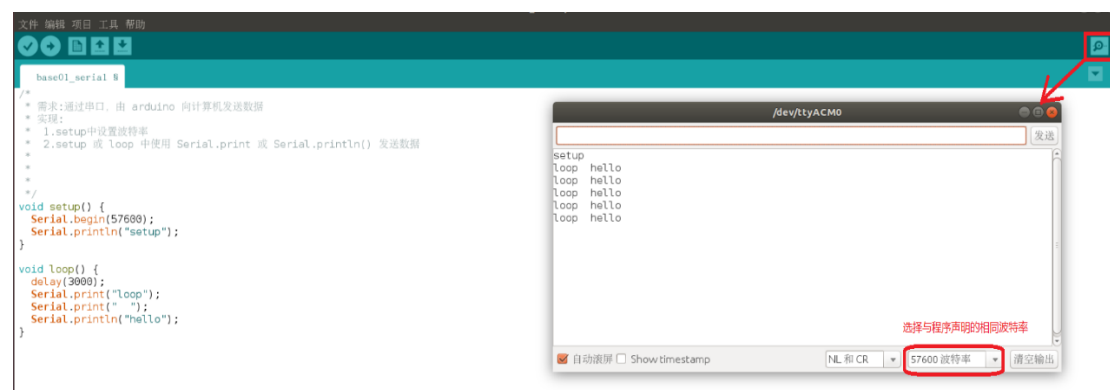
1.1. 通信实现 01（由 Arduino 向计算机发送数据）

需求：通过串口，由 arduino 向计算机发送数据

实现：

```
/*
 * 需求:通过串口, 由 arduino 向计算机发送数据
 * 实现:
 * 1.setup 中设置波特率
 * 2.setup 或 loop 中使用 Serial.print 或 Serial.println() 发送数据
 */
void setup() {
    Serial.begin(57600);
    Serial.println("setup");
}

void loop() {
    delay(3000);
    Serial.print("loop");
    Serial.print(" ");
    Serial.println("hello");
}
```



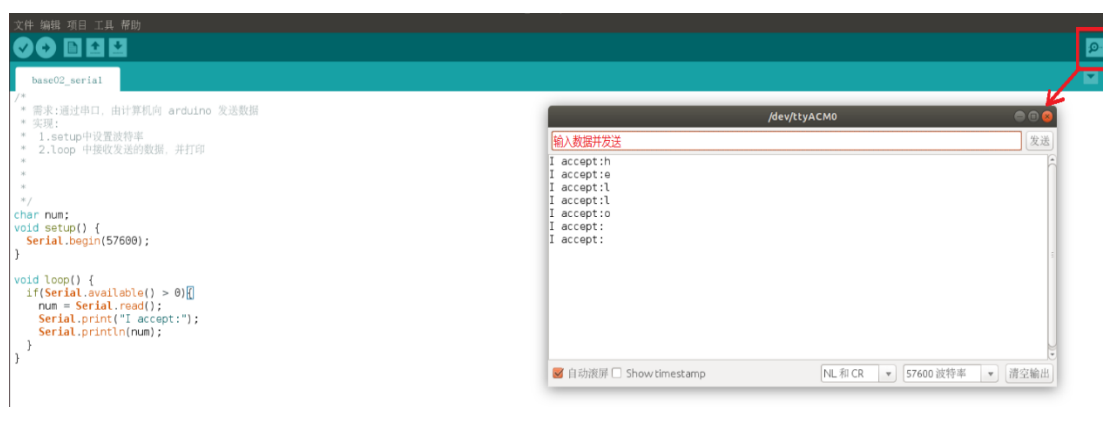
1.2. 通信实现 02（由计算机向 Arduino 发送数据）

需求：通过串口，由计算机向 Arduino 发送数据

实现：

```
/*
 * 需求:通过串口, 由计算机向 arduino 发送数据
 * 实现:
 * 1.setup 中设置波特率
 * 2.loop 中接收发送的数据, 并打印
 */
char num;
```

```
void setup() {  
  Serial.begin(57600);  
}  
  
void loop() {  
  if(Serial.available() > 0){  
    num = Serial.read();  
    Serial.print("I accept:");  
    Serial.println(num);  
  }  
}
```



2. IO 操作

2.1. 数字 IO 操作

需求:控制 LED 灯开关, 在一个循环周期内前两秒使 LED 灯处于点亮状态, 后两秒关闭 LED 灯

实现:

```
/*  
 * 控制 LED 灯开关, 在一个循环周期内前两秒使 LED 灯处于点亮状态, 后两秒关闭 LED 灯  
 * 1.setup 中设置引脚为输出模式  
 * 2.loop 中向引脚输出高电压, 休眠 2000 毫秒后, 再输出低电压, 再休眠 2000 毫秒  
 */  
int led = 13;  
void setup() {  
  Serial.begin(57600);  
  pinMode(led, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(led, HIGH); //输出高电压
```

```
delay(2000);
```

```
digitalWrite(led,LOW);//输出低电压
```

```
delay(2000);
```

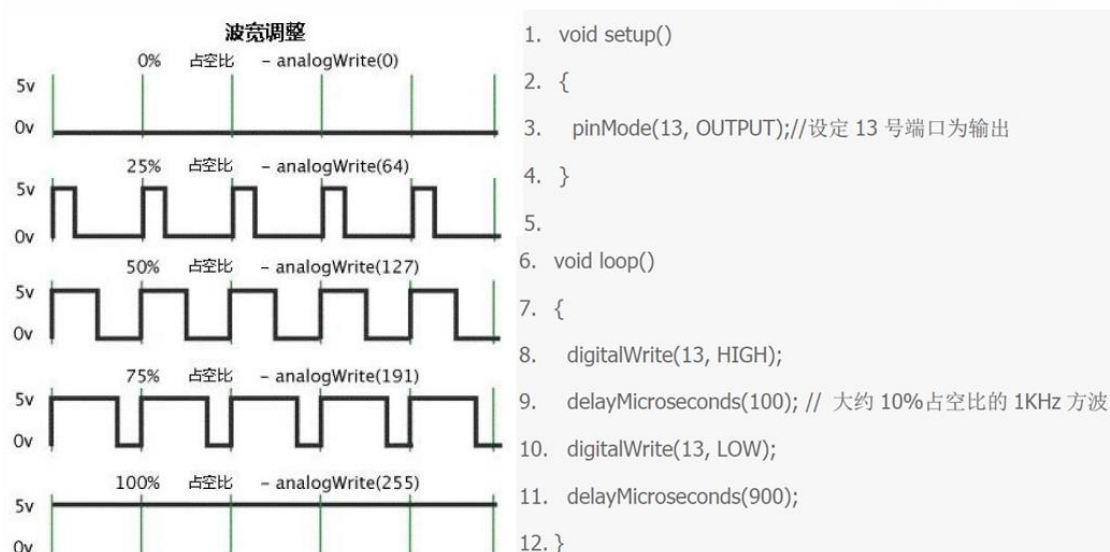
```
}
```

2.2. 模拟 IO 操作

需求:控制 LED 灯亮度

原理:在 1 中 LED 灯只有关闭或开启两种状态,是无法控制 LED 灯亮度,如果要实现此功能,那么需要借助于 PWM(Pulse width modulation 脉冲宽度调制)技术,通过设置占空比为 LED 间歇性供电, PWM 的取值范围 [0, 255]。

● PWM (Pulse-width modulation) 脉宽调制



`analogWrite()`:模拟 I/O 口输出,一般用于 PWM 输出,如: `analogWrite(13,127)`,为在 13 号引脚处输出一个占空比为 50%的 PWM 方波,后一参数 0 表示 关, 255 表示全开

实现:

```
/*
 * 需求:控制 LED 灯亮度
 * 实现:
 * 1.setup 中设置 led 灯的引脚为输出模式
 * 2.设置不同的 PWM 并输出
 */
int led = 13;
int l1 = 255;
int l2 = 50;
int l3 = 0;
void setup() {
  pinMode(led,OUTPUT);
}
```

```
void loop() {  
    analogWrite(led,11);  
    delay(2000);  
    analogWrite(led,12);  
    delay(2000);  
    analogWrite(led,13);  
    delay(2000);  
}
```

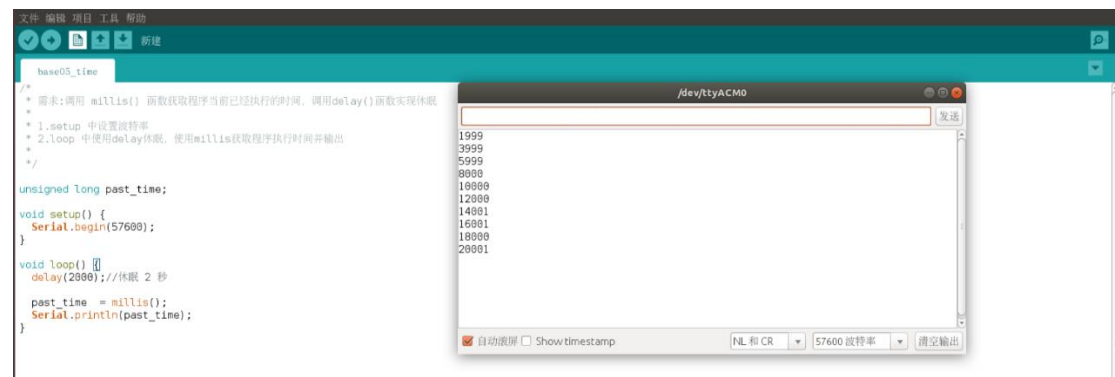
运行结果:在一个周期内 LED 灯亮度递减直至熄灭

3. 调用 millis() 函数

需求:调用 millis() 函数获取程序当前已经执行的时间,调用 delay() 函数实现休眠实现:

```
/*  
 * 需求:调用 millis() 函数获取程序当前已经执行的时间,调用 delay()函数  
实现休眠  
 *  
 * 1.setup 中设置波特率  
 * 2.loop 中使用 delay 休眠,使用 millis()获取程序执行时间并输出  
 */  
  
unsigned long past_time;  
  
void setup() {  
    Serial.begin(57600);  
}  
  
void loop() {  
    delay(2000); //休眠 2 秒  
  
    past_time = millis();  
    Serial.println(past_time);  
}
```

通过串口监视器查看输出结果。



六、Arduino 与直流电机驱动（以 ROS 智能两轮小车为例）

1. 硬件-电机与电机驱动板

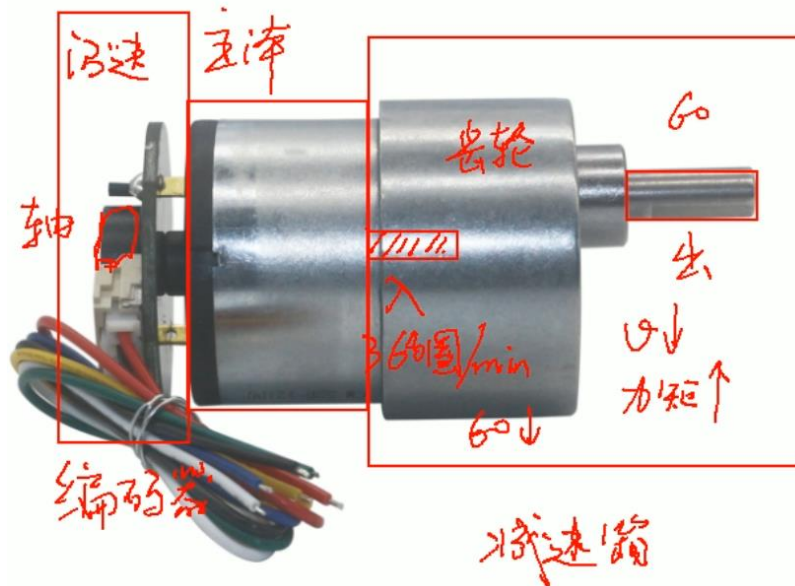
如果要通过 Arduino 实现电机相关操作(比如:转向控制、转速控制、测速等),那么必须先要具备两点前提知识:

1. 需要简单了解电机类型、机械结构以及各项参数,这些是和机器人的负载、极限速度、测速结果等休戚相关的;
2. 还需要选配合适的电机驱动板,因为 Arduino 的输出电流不足以直接驱动电机,需要通过电机驱动板放大电机控制信号。

当前我们的机器人平台使用的电机为直流减速电机,电机驱动板为基于 L298P 实现的电路板。接下来就分别介绍这两个模块:

1.1. 直流减速电机

如图所示,相当一部分 ROS 智能车中使用的直流减速电机与之类似,主要由三部分构成:



- 减速箱
- 电机主体
- 编码器

电机主体通过输入轴与减速箱相连接,通过减速箱的减速效果,最终外端的输出轴会按照比例(取决于减速箱减速比)降低电机输入轴的转速,当然速度降低之后,将提升电机的力矩。

尾部是 AB 相霍尔编码器,通过 AB 编码器输出的波形图,可以判断电机的转向以及计算电机转速

另外,即便电机外观相同,具体参数也可能存在差异,参数需要商家提供,需要了解的参数如下:

- 额定电压
- 额定电流
- 额定功率
- 额定扭矩
- 减速比
- 减速前转速

- 减速后转速
- 编码器精度

主要参数:

额定扭矩:额定扭矩和机器人质量以及有效负荷相关,二者正比例相关,额定扭矩越大,可支持的机器人质量以及有效负荷越高;

减速比:电机输入轴与输出轴的减速比例,比如:减速比为 90,意味着电机主体旋转 90 圈,输出轴旋转 1 圈。

减速后转速:与减速比相关,是电机减速箱输出轴的转速,单位是 rpm(转/分),减速后转速与减速前转速存在转换关系: $\text{减速后转速} = \text{减速前转速} / \text{减速比}$ 。另外,可以根据官方给定的额定功率下的减速后转速结合车轮参数来确定小车最大速度。

编码器精度:是指编码器旋转一圈单相(当前编码器有 AB 两相)输出的脉冲数;

注意:电机输入轴旋转一圈的同时,编码器旋转一圈,如果输出轴旋转一圈,那么编码器的旋转圈数和减速比一致(比如减速比是 90,那么输出轴旋转一圈,编码器旋转 90 圈)。

编码器输出的脉冲数计算公式则是: $\text{输出轴旋转一圈产生的脉冲数} = \text{减速比} * \text{编码器旋转一圈发送的脉冲数}$ (比如:减速比为 90,编码器旋转一圈输出 11 个脉冲,那么输出轴旋转一圈总共产生 $11 * 90$ 也即 990 个脉冲)。

电机编码器



M1: 电机电源+(和 M2 对调可以正反转)

GND: 编码器电源-

C2: 信号线

C1: 信号线

VCC: 编码器电源+

M2: 电机电源-(和 M1 对调可以正反转)

1.2. 电机驱动板

电机驱动板可选型号较多,比如:TB6612、L298N、L298P...但是这些电机驱动板与电机相连时,需要使用杜邦线,接线会显得凌乱,本节会采用一款基于 L298P 优化的电机驱动板,该驱动板可以使用端子线直接连接电机,接线更规整、美观。



| 端子线母头对应的引脚(自上而下)

| 母头 1: 4、地线、21、20、5V 输入、5

| 母头 2: 7、地线、18、19、5V 输入、6

PS: 电机驱动板使用时, 需要打开 USB 接口处的电源开关。

1.3. 准备工作

组装底盘: 集成电池、Arduino、电机驱动板与电机



先安装 Arduino、安装电机(接端子线)与万向轮, 将电机驱动板与 Arduino 集成;

然后将电池的正负极分别接入电机驱动模块的 12V 与 GND(注意:正负极不可接反, 12V 接红线, GND 接黑线);

最后将电机通过端子线与驱动板相连。

2. 直流电机基本控制实现

在 ROS 智能车中，控制车辆的前进、后退以及速度调节，那么就涉及到电机的转向与转速控制，本节主要就是介绍相关知识。

2.1. 需求

控制单个电机转动，先控制电机以某个速率正向转动 N 秒，再让电机停止 N 秒，再控制电机以某个速率逆向转动 N 秒，最后让电机停止 N 秒，如此循环。

2.2. 实现流程

1. 编写 Arduino 程序，setup 中设置引脚模式，loop 中控制电机运动；

2. 上传并查看运行结果。

2.3. 编码

前提知识点：

1. 左电机的 M1 与 M2 对应的是引脚 4 (DIRA) 和引脚 5 (PWMA)，引脚 4 控制转向，引脚 5 输出 PWM。右电机的 M1 与 M2 对应的是引脚 6 (PWMB) 和引脚 7 (DIRB)，引脚 7 控制转向，引脚 6 输出 PWM。
2. 可以通过 PWM 控制电机转速。

代码：

```
/*
 * 电机转动控制
 * 1. 定义接线中电机对应的引脚
 * 2. setup 中设置引脚为输出模式
 * 3. loop 中控制电机转动
 */

int DIRA = 4;
int PWMA = 5;

void setup() {
  //两个引脚都设置为 OUTPUT
  pinMode(DIRA, OUTPUT);
  pinMode(PWMA, OUTPUT);
}

void loop() {
  //先正向转动 3 秒
  digitalWrite(DIRA, HIGH);
  analogWrite(PWMA, 100);
  delay(3000);
  //停止 3 秒
  digitalWrite(DIRA, HIGH);
  analogWrite(PWMA, 0);
  delay(3000);
  //再反向转动 3 秒
  digitalWrite(DIRA, LOW);
  analogWrite(PWMA, 100);
  delay(3000);
}
```

```
//停止 3 秒
digitalWrite(DIRA,LOW);
analogWrite(PWMA,0);
delay(3000);
/*
 * 注意:
 * 1.可以通过将 DIRA 设置为 HIGH 或 LOW 来控制电机转向,但是哪个标志位
正转或反转根据需求判断,转向是相对的。
 * 2.PWM 的取值为 [0,255],该值可自己设置。
 */
}
```

2.4. 运行

程序上传到 Arduino 上,如无异常,电机开始转动,转动结果与需求描述类似。

3. 直流电机测速 (理论部分)

测速实现是调速实现的前提,本节主要介绍 AB 相增量式编码器测速原理。

3.1. 概念

百度百科关于编码器介绍如下:

编码器 (encoder) 是将信号 (如比特流) 或数据进行编制、转换为可用以通讯、传输和存储的信号形式的设备。

编码器把角位移或直线位移转换成电信号,前者称为码盘,后者称为码尺。

按照读出方式编码器可以分为接触式和非接触式两种;按照工作原理编码器可分为增量式和绝对式两类。

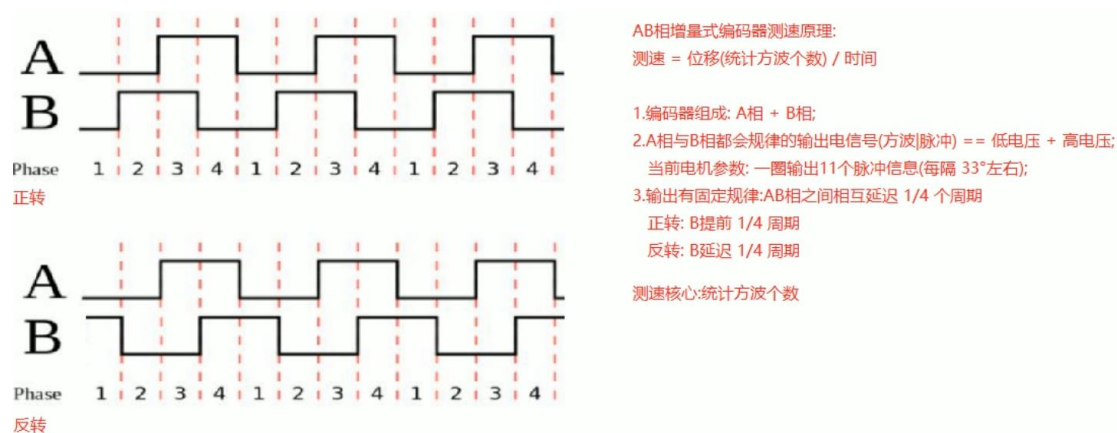
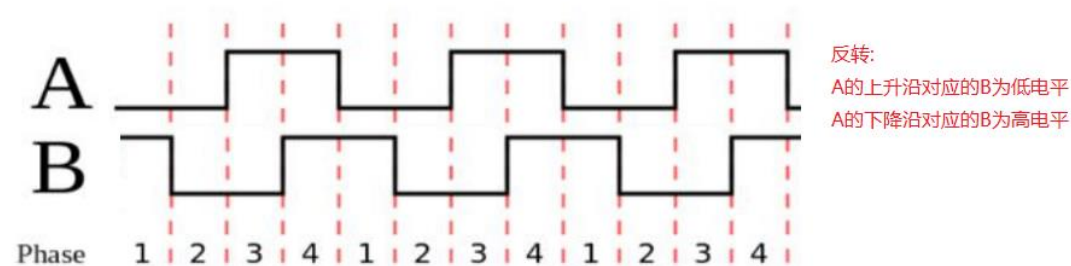
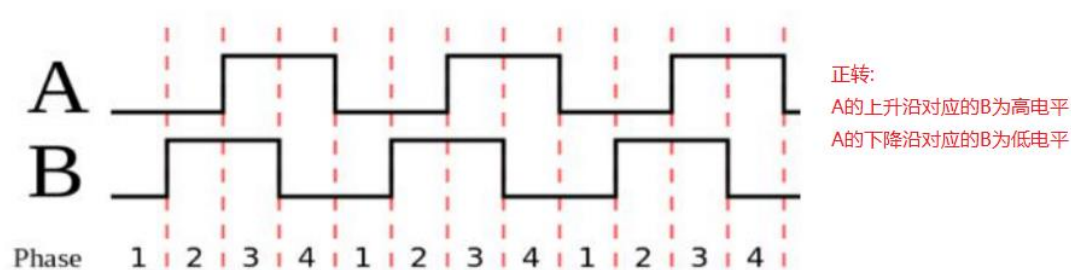
增量式编码器是将位移转换成周期性的电信号,再把这个电信号转变成计数脉冲,用脉冲的个数表示位移的大小。

绝对式编码器的每一个位置对应一个确定的数字码,因此它的示值只与测量的起始和终止位置有关,而与测量的中间过程无关。

3.2. 测速原理

关于编码器相关概念简单了解即可,在此需要着重介绍的是 AB 相增量式编码器测速原理:

AB 相编码器主要构成为 A 相与 B 相,每一相每转过单位的角度就发出一个脉冲信号 (一圈可以发出 N 个脉冲信号),A 相、B 相为相互延迟 1/4 周期的脉冲输出,根据延迟关系可以区别正反转,而且通过取 A 相、B 相的上升和下降沿可以进行单频或 2 倍频或 4 倍频测速。



3.3. 测速举例

假设编码器旋转 1 圈输出 11 个脉冲，减速比为 90。伪代码如下：

单频计数：

```
// 设置一个计数器
int count = 0;
// 当 A 为上升沿时
if(B 为高电平){
    count++;
}else {
    count--;
}
// ...
// 速度 = 单位时间内统计的脉冲的个数 / (11*90) / 单位时间
```

2 倍频计数：

```
// 设置一个计数器
int count = 0;
// 当 A 为上升沿时
```



```
if(B 为高电平){  
    count++;  
}else {  
    count--;  
}  
//当 A 为下降沿时  
if(B 为低电平){  
    count++;  
}else {  
    count--;  
}  
  
//....  
//速度=单位时间内统计的脉冲的个数 / (11*2*90) / 单位时间
```

4 倍频计数:

```
//设置一个计数器  
int count = 0;  
//当 A 为上升沿时  
if(B 为高电平){  
    count++;  
}else {  
    count--;  
}  
//当 A 为下降沿时  
if(B 为低电平){  
    count++;  
}else {  
    count--;  
}  
//当 B 为上升沿时  
if(A 为低电平){  
    count++;  
} else {  
    count--;  
}  
//当 B 为下降沿时  
if(A 为高电平){  
    count++;  
} else {  
    count--;  
}  
//....  
//速度=单位时间内统计的脉冲的个数 / (11*4*90) / 单位时间
```

4. 直流电机测速（代码实现部分）

需求:统计并输出电机转速。

思路:先统计单位时间内以单频或 2 倍频或 4 倍频的方式统计脉冲数，再除以一圈对应的脉冲数，最后再除以时间所得即为电机转速。

核心:计数时，需要在 A 相或 B 相的上升沿或下降沿触发时，实现计数，在此需要使用中断引脚与中断函数。

Arduino Mega 2560 的中断引脚:2 (interrupt 0), 3 (interrupt 1),18 (interrupt 5), 19 (interrupt 4), 20 (interrupt 3), 21 (interrupt 2)

实现流程:

1. 编写 Arduino 程序先实现脉冲数统计;
2. 写 Arduino 程序再实现转速计算相关实现;
3. 上传到 Arduino 并测试。

4.1. 编码实现脉冲统计

核心知识点:**attachInterrupt()** 函数(请参考 第四章的 7. 函数_中断 介绍)。

代码:

```
/*
 * 测速实现:
 * 阶段 1:脉冲数统计
 * 阶段 2:速度计算
 *
 * 阶段 1:
 * 1.定义所使用的中断引脚,以及计数器(使用 volatile 修饰)
 * 2.setup 中设置波特率,将引脚设置为输入模式
 * 3.使用 attachInterrupt() 函数为引脚添加中断出发时机以及中断函数
 * 4.中断函数编写计算算法,并打印
 *   A.单频统计只需要统计单相上升沿或下降沿
 *   B.2 倍频统计需要统计单相的上升沿和下降沿
 *   C.4 倍频统计需要统计两相的上升沿和下降沿
 * 5.上传并查看结果
 */
int motor_A = 21;//中端口是 2
int motor_B = 20;//中断口是 3
volatile int count = 0;//如果是正转,那么每计数一次自增 1,如果是反转,
那么每计数一次自减 1

void count_A(){
  //单频计数实现
  //手动旋转电机一圈,输出结果为 一圈脉冲数 * 减速比
  /*if(digitalRead(motor_A) == HIGH){
    if(digitalRead(motor_B) == LOW){//A 高 B 低
      count++;
    } else {//A 高 B 高
      count--;
    }
  }
  */
}
```



```
    */

    //2 倍频计数实现
    //手动旋转电机一圈，输出结果为 一圈脉冲数 * 减速比 * 2
    if(digitalRead(motor_A) == HIGH){
        if(digitalRead(motor_B) == HIGH){//A 高 B 高
            count++;
        } else {//A 高 B 低
            count--;
        }
    } else {
        if(digitalRead(motor_B) == LOW){//A 低 B 低
            count++;
        } else {//A 低 B 高
            count--;
        }
    }
}

//与 A 实现类似
//4 倍频计数实现
//手动旋转电机一圈，输出结果为 一圈脉冲数 * 减速比 * 4
void count_B(){
    if(digitalRead(motor_B) == HIGH){
        if(digitalRead(motor_A) == LOW){//B 高 A 低
            count++;
        } else {//B 高 A 高
            count--;
        }
    } else {
        if(digitalRead(motor_A) == HIGH){//B 低 A 高
            count++;
        } else {//B 低 A 低
            count--;
        }
    }
}

void setup() {
    Serial.begin(57600);//设置波特率
    pinMode(motor_A,INPUT);
    pinMode(motor_B,INPUT);
    attachInterrupt(2,count_A,CHANGE);//当电平发生改变时触发中断函数
    //四倍频统计需要为 B 相也添加中断
```

```

attachInterrupt(3,count_B,CHANGE);
}

void loop() {
  //测试计数器输出
  delay(2000);
  Serial.println(count);
}

```

4.2. 转速计算

思路:需要定义一个开始时间(用于记录每个测速周期的开始时刻),还需要定义一个时间区间(比如 50 毫秒),时时获取当前时刻,当 当前时刻 - 上传结束时刻 >= 时间区间时,就获取当前计数并根据测速公式计算时时速度,计算完毕,计数器归零,重置开始时间

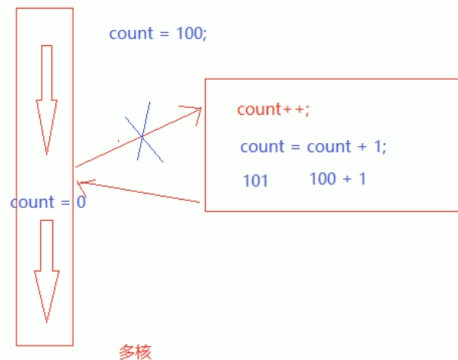
核心知识点:当使用中断函数中的变量时,需要先禁止中断 `noInterrupts()`,调用完毕,再重启中断 `interrupts()` (关于 `noInterrupts` 与 `interrupts` 请参考 第四章的 7. 函数_中断 介绍)。(避免多线程的安全隐患,但 Arduino 本身是单核的,不存在多核执行的情况,所以不加这两个也没事)

```

void get_current_vel(){
  //获取时时时间戳(当前时间)
  long right_now = millis();
  //判断逝去的时间是否大于单位时间
  long past_time = right_now - start_time;
  if(past_time >= interval_time){
    //取消中断
    noInterrupts(); //中断函数不再执行,无 count++ 或 count-- 操作。
    //计算转速(count)
    double vel = (double)count / per_round / past_time * 1000 * 60;
    Serial.println(vel);
    //将count置0
    count = 0;
    //将开始时间重置为当前时间
    start_time = right_now;
    //重启中断
    interrupts();
  }
}

```

假设没有取消中断,会怎样?



代码(核心):

4.2 中代码除了 `loop` 实现,无需修改。

```

int reduction = 90; //减速比,根据电机参数设置,比如 15 | 30 | 60
int pulse = 11; //编码器旋转一圈产生的脉冲数,该值需要参考商家电机参数
int per_round = pulse * reduction * 4; //车轮旋转一圈产生的脉冲数
long start_time = millis(); //一个计算周期的开始时刻,初始值为
millis();

long interval_time = 50; //一个计算周期 50ms
double current_vel;

```

//获取当前转速的函数

```

void get_current_vel(){
  long right_now = millis();
  long past_time = right_now - start_time; //计算逝去的时间
  if(past_time >= interval_time){ //如果逝去时间大于等于一个计算周期
    //1. 禁止中断
    noInterrupts();

```

```
//2.计算转速 转速单位可以是秒，也可以是分钟... 自定义即可（这里是圈数/分钟）
current_vel = (double)count / per_round / past_time * 1000 * 60;

//3.重置计数器
count = 0;
//4.重置开始时间
start_time = right_now;
//5.重启中断
interrupts();
Serial.println(current_vel);
}
}

void loop() {
    delay(10);
    get_current_vel();
}
```

4.3. 测试

将代码上传至 Arduino，打开串口监视器，手动旋转电机，可以查看到转速信息。

5. 直流电机调速（PID 控制理论部分）

| 场景：

| 速度信息可以以 m/s 为单位，或者也可以转换成转速 r/s，而电机的转速是由 PWM 脉冲宽度来控制的，如何根据速度信息量化成合适的 PWM 值呢？

| 比如：现有一辆行驶中的无人车，要求将车速调整至 100KM/h，那么应该如何向电机输出 PWM 值？或换言之，如何控制油门？

调速实现策略由多种，PID 其中较为常用。

【PID 简介】

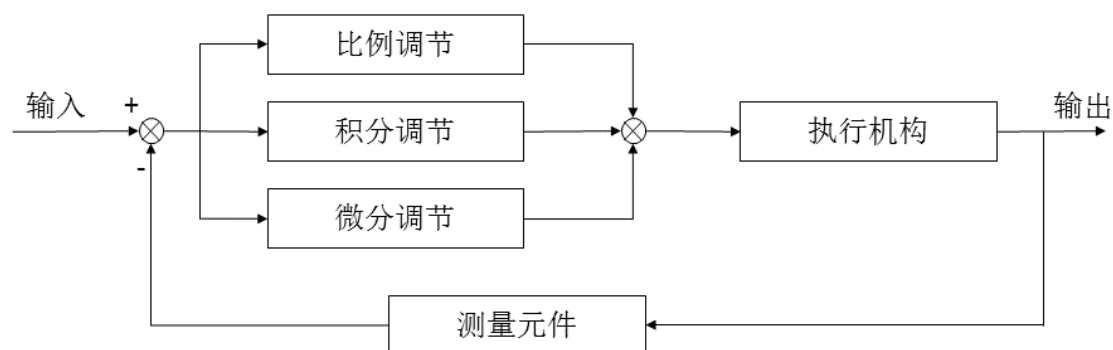
PID 算法是一种经典、简单、高效的动态速度调节方式，P 代表比例，I 代表积分，D 代表微分。

PID 公式如下：

$$\mu(t) = K_P e(t) + K_I \int_0^t e(t) dt + K_D \frac{de(t)}{dt}$$

- $e(t)$ 作为 PID 控制的输入；
- $u(t)$ 作为 PID 控制器的输出和被控对象的输入；
- K_P 控制器的比例系数；
- K_I 控制器的积分时间，也称积分系数；
- K_D 控制器的微分时间，也称微分系数。

上述公式稍显晦涩，PID 控制原理框架图更有助于理解：



需求:将车速从 0 加速稳定在 100km/h, 怎么控制油门(输出PWM)?

答:使用PID算法。

狠(P:比例) ---- 尽快的到达目标值;

准(I:积分) ---- 大限度的接近目标值;

稳(D:微分) ---- 控制目标值的波动范围。

5.1.P

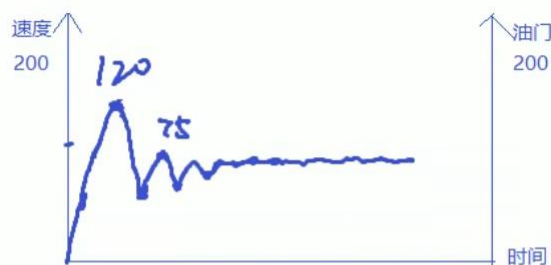
如果实现上述场景中的车速控制,一种简单的实现方式是:确定目标速度,获取当前速度,使用(目标速度-当前速度)*某一系数 计算结果为输出的 PWM,再获取当前速度,使用(目标速度-当前速度)*某一系数 计算结果为输出的 PWM 并输出...如此循环在上述模型中,调速实现是一个闭环,每一次循环都会根据当前时速与目标时速的差值,再乘以以固定系数,计算出需要输出的 PWM 值,这其中的系数,称之为比例。

P(比例) 怎么实现的?

1.P控速时, 是一个循环过程;

2.通过P可以输出PWM值,公式: $P * (\text{目标速度} - \text{当前速度})$

P调试过程以及结果图示:



调试结果: 最终波动会越来越小, 并最终到达一个稳态状态。

稳态值是多少?

$P * (\text{目标速度} - \text{稳态速度}) = \text{PWM}(\text{PWM刚好能够达成稳态速度})$

设: $v:p = 1:1$

$2 * (100 - v) = v$

$200 - 2v = v$

$v = 66.66$

结果:到达稳定状态后, 最终速度始终达不到目标速度, 会产生一个稳态误差

设: $v:p = 1:1$

$x * (100 - v) = v;$

$100x - xv = v$

$100x = (x + 1)v$

$v = 100x / (x + 1)$

5.2. I

上述模型算法中, 最终速度与预期速度存在稳态误差, 这意味着最终结果可能永远无法达成预期, 解决的方法就是使用积分 I。每次调速时, 输出的 PWM 还要累加根据积分 I 计算的结果, 以消除静态误差。

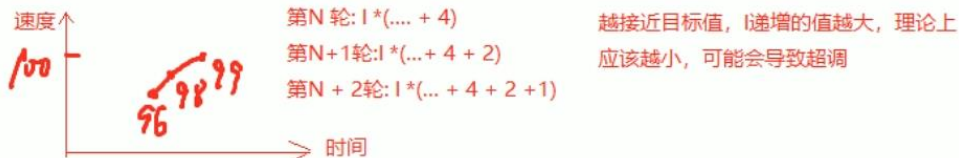
I(积分)怎么实现?

1. 每次循环执行 P 运算时, 会累加运算的结果;

2. I 基本算法: $I * (\text{误差1} + \text{误差2} + \dots)$

I 的计算过程: $I * (-20 + 40 + 25 \dots)$

I 的计算结果: 超调并且会震荡



5.3. D

当 I 值设置的过大时, 可能会出先"超速"的情况, 超速之后可能需要多次调整, 产生系统震荡, 解决这种情况可以使用 D 微分, 当速度越是接近目标速度时, D 就会越施加反方向力, 减弱 P 的控制, 起到类似"阻尼"的作用。通过 D 的使用可以减小系统震荡。

D(微分)怎么实现?

1. 每次循环执行 P 和 I 之后, 都会累加 D 的运算结果;

2. D 基本算法: $D * (\text{本次误差} - \text{上次误差})$ 。

D 的计算过程:

D 的计算: $D * (2 - 4) = -2D$

$D * (1 - 2) = -D$

现象: 当接近目标值时, I 会越来越大, D 可以减小 I 的增益,

起到了阻尼的作用, 最终的效果: 越接近目标值时, 增加或减

小的 PWM 幅度趋于放缓, 以减小震荡。

综上, PID 闭环控制实现是结合了比例、积分和微分的一种控制机制, 通过 P 可以以比例的方式计算输出, 通过 I 可以消除稳态误差, 通过 D 可以减小系统震荡, 三者相结合, 最终是要快速、精准且稳定的达成预期结果, 而要实现该结果, 还需要对这三个数值反复测试、调整... 下一节将介绍在 Arduino 中 PID 控制的具体实现, 其中就包括 PID 库的调用, 以及 PID 调试的具体方式。

6. 直流电机调速 (PID 控制实践部分)

了解了 PID 原理以及计算公式之后, 我们可以在程序中自实现 PID 相关算法实现, 不过, 在 Arduino 中该算法已经被封装了, 直接整合调用即可, 从而提高程序的安全性与开发效率。该库是: Arduino-PID-Library, 接下来通过一个案例演示该库的使用。

需求: 通过 PID 控制电机转速, 预期转速为 80r/m。

实现流程:

1. 添加 Arduino-PID-Library;
2. 编写 Arduino 程序直接调用相关实现;
3. 使用串口绘图器调试 PID 值。

6.1. 添加 Arduino-PID-Library

(这是 Linux 版)

(1) 首先在 GitHub 下载 PID 库:

```
git clone https://github.com/br3ttb/Arduino-PID-Library
```

(2) 然后将该文件夹移动到 arduino 的 libraries 下:

```
sudo cp -r Arduino-PID-Library /home/用户 xxx/Arduino/libraries
```

(3) 还要重命名文件夹:

```
sudo mv Arduino-PID-Library ArduinoPIDLibrary
```

(4) 最后重启 ArduinoIDE

(这是 Windows 版)

(1) 首先在该网址下载 PID 库: <https://github.com/br3ttb/Arduino-PID-Library>

(2) 然后将该文件夹(Arduino-PID-Library)移动到 Arduino 的 libraries 下

(3) 将 Arduino-PID-Library 重命名为 ArduinoPIDLibrary

(4) 最后重启 Arduino IDE

6.2. 编码

PID 调速中, 测速是实现闭环的关键实现, 所以需要复制之前的电机控制代码以及测速代码。

完整代码实现:

```
/*
 * PID 调速实现:
 * 1. 代码准备, 复制并修改电机控制以及测速代码
 * 2. 包含 PID 头文件
 * 3. 创建 PID 对象
 * 4. 在 setup 中启用自动调试
 * 5. 调试并更新 PWM
 */

#include <PID_v1.h>

int DIRA = 4;
int PWMA = 5;

int motor_A = 21; // 中端口是 2
int motor_B = 20; // 中断口是 3
volatile int count = 0; // 如果是正转, 那么每计数一次自增 1, 如果是反转,
那么每计数一次自减 1

void count_A(){
    // 单频计数实现
    // 手动旋转电机一圈, 输出结果为 一圈脉冲数 * 减速比
    /*if(digitalRead(motor_A) == HIGH){
        if(digitalRead(motor_B) == LOW){ // A 高 B 低
            count++;
        } else { // A 高 B 高
            count--;
        }
    }
    */

    // 2 倍频计数实现
```

```
//手动旋转电机一圈，输出结果为 一圈脉冲数 * 减速比 * 2
if(digitalRead(motor_A) == HIGH){
    if(digitalRead(motor_B) == HIGH){//A 高 B 高
        count++;
    } else {//A 高 B 低
        count--;
    }
} else {
    if(digitalRead(motor_B) == LOW){//A 低 B 低
        count++;
    } else {//A 低 B 高
        count--;
    }
}
}
```

//与 A 实现类似

//4 倍频计数实现

//手动旋转电机一圈，输出结果为 一圈脉冲数 * 减速比 * 4

```
void count_B(){
    if(digitalRead(motor_B) == HIGH){
        if(digitalRead(motor_A) == LOW){//B 高 A 低
            count++;
        } else {//B 高 A 高
            count--;
        }
    } else {
        if(digitalRead(motor_A) == HIGH){//B 低 A 高
            count++;
        } else {//B 低 A 低
            count--;
        }
    }
}
```

int reduction = 90;//减速比，根据电机参数设置，比如 15 | 30 | 60

int pulse = 11; //编码器旋转一圈产生的脉冲数，该值需要参考商家电机参数

int per_round = pulse * reduction * 4;//车轮旋转一圈产生的脉冲数

long start_time = millis();//一个计算周期的开始时刻，初始值为 millis();

long interval_time = 50;//一个计算周期 50ms

double current_vel;

//获取当前转速的函数


```
void get_current_vel(){
    long right_now = millis();
    long past_time = right_now - start_time;//计算逝去的时间
    if(past_time >= interval_time){//如果逝去时间大于等于一个计算周期
        //1.禁止中断
        noInterrupts();
        //2.计算转速 转速单位可以是秒，也可以是分钟... 自定义即可
        current_vel = (double)count / per_round / past_time * 1000 *
60;
        //3.重置计数器
        count = 0;
        //4.重置开始时间
        start_time = right_now;
        //5.重启中断
        interrupts();
        Serial.println(current_vel);
    }
}

//-----PID-----
---
//创建 PID 对象
//1.当前转速 2.计算输出的 pwm 3.目标转速 4.kp 5.ki 6.kd 7.当输入与目
标值出现偏差时，向哪个方向控制
double pwm;//电机驱动的 PWM 值
double target = 80;
double kp=1.5, ki=3.0, kd=0.1;
PID pid(&current_vel,&pwm,&target,kp,ki,kd,DIRECT);

//速度更新函数
void update_vel(){
    //获取当前速度
    get_current_vel();
    pid.Compute();//计算需要输出的 PWM
    digitalWrite(DIR_A,HIGH);
    analogWrite(PWMA,pwm);
}

void setup() {
    Serial.begin(57600);//设置波特率
    pinMode(18,INPUT);
    pinMode(19,INPUT);
    //两个电机驱动引脚都设置为 OUTPUT
    pinMode(DIR_A,OUTPUT);
}
```

```
pinMode(PWMA,OUTPUT);

attachInterrupt(2,count_A,CHANGE);//当电平发生改变时触发中断函数
//四倍频统计需要为 B 相也添加中断
attachInterrupt(3,count_B,CHANGE);

pid.SetMode(AUTOMATIC);
}

void loop() {
    delay(10);
    update_vel();
}
}
```

核心代码解释:

1. 包含 PID 头文件

```
#include <PID_v1.h>
```

2. 创建 PID 对象

```
//创建 PID 对象
```

//1.当前转速 2.计算输出的 pwm 3.目标转速 4.kp 5.ki 6.kd 7.当输入与目标值出现偏差时, 向哪个方向控制(DIRCET | REVERSE)

```
double pwm;//电机驱动的 PWM 值
```

```
double target = 120;
```

```
double kp=1.5, ki=3.0, kd=0.1;
```

```
PID pid(&current_vel,&pwm,&target,kp,ki,kd,DIRECT);
```

3. setup 中启用 PID 自动控制

```
pid.SetMode(AUTOMATIC);
```

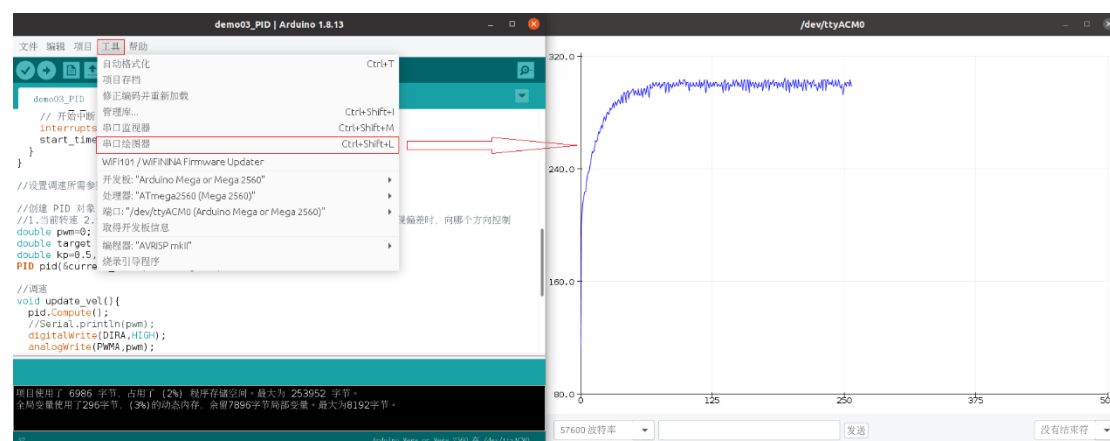
4. 计算输出值

```
pid.Compute();
```

6.3. 调试

PID 控制的最终预期结果, 是要快速、精准、稳定的达成预期结果, P 主要用于控制响应速度, I 主要用于控制精度, D 主要用于减小震荡增强系统稳定性, 三者的取值是需要反复调试的, 调试过程中需要查看系统的响应曲线, 根据响应曲线以确定合适的 PID 值。

在 Arduino 中响应曲线的查看可以借助于 Serial.println() 将结果输出, 然后再选择菜单栏的工具下串口绘图器以图形化的方式显示响应结果:



6.3. PID 调试口诀

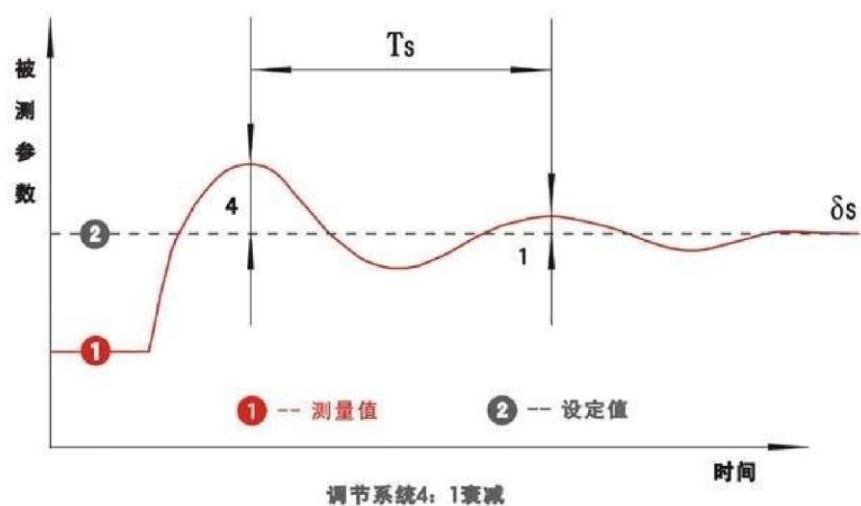
参数整定找最佳，从小到大顺序查
先是比例后积分，最后再专把微分加
曲线振荡很频繁，比例度盘要放大
曲线漂浮绕大湾，比例度盘往小扳
曲线偏离回复慢，积分时间往下降
曲线波动周期长，积分时间再加长
曲线振荡频率快，先把微分降下来
动差大来波动慢，微分时间应加长
理想曲线两个波，前高后低4比1
一看二调多分析，调节质量不会低

【附：口诀详解】

PID 调试口诀是一系列指导原则，用于调整 **PID 控制器** 的参数，以确保系统的稳定性和性能。这些口诀基于经验和观察，提供了一种直观且有效的方法来调整 PID 控制器的比例（P）、积分（I）和微分（D）参数。以下是每个口诀的详细解释：

1. 参数整定找最佳，从小到大顺序查：这意味着在调整 PID 参数时，应从小到大逐步调整每个参数，以找到最佳的控制效果。
2. 先是比例后积分，最后再把微分加：建议首先调整比例参数，然后是积分参数，最后是微分参数。这是因为比例参数直接影响系统的响应速度和稳定性，积分参数用于消除稳态误差，而微分参数则用于减少系统的超调和振荡。
3. 曲线振荡很频繁，比例度盘要放大：当系统出现频繁的振荡时，应增加比例度（减小比例增益），以减少振荡频率和幅度。
4. 曲线漂浮绕大湾，比例度盘往小扳：如果系统的响应曲线偏离设定值较大，应减小比例度（增加比例增益），以加快响应速度并减少偏差。
5. 曲线偏离回复慢，积分时间往下降：当系统对设定值的响应较慢时，应减小积分时间，以加快系统的响应速度。
6. 曲线波动周期长，积分时间再加长：如果系统的响应曲线波动周期较长，应增加积分时间，以减少稳态误差和提高系统的稳定性。
7. 曲线振荡频率快，先把微分降下来：当系统出现快速振荡时，应减小微分时间（或关闭微分作用），以减少系统的超调和振荡。
8. 动差大来波动慢，微分时间应加长：当系统动态误差较大时，应增加微分时间，以提高系统的稳定性和减少超调。
9. 理想曲线两个波，前高后低4比1：指下图的曲线，也就是过渡过程振荡两次就能稳

定下来，并且振荡两次后有近于 4: 1 的衰减比，它被认为是最好的过渡过程。



七、Arduino 与底盘实现

在 `ros` 中还提供了一个已经封装了的模块：`ros_arduino_bridge`，该模块由下位机驱动和上位机控制两部分组成，通过该模块可以更为快捷、方便的实现自己的机器人平台。本节介绍的主要内容如下：

- `ros_arduino_bridge` 的架构；
- 下位机端 `Arduino` 程序修改。

注意：官方提供的案例所需硬件国内不易购买，需要修改源码适配当前硬件。

1. 概述

1.1. `ros_arduino_bridge` 简介

该功能包包含 `Arduino` 库和用来控制 `Arduino` 的 `ROS` 驱动包，它旨在成为在 `ROS` 下运行 `Arduino` 控制的机器人的完整解决方案。

其中当前主要关注的是：功能包集中一个兼容不同驱动的机器人的基本控制器（`base controller`），它可以接收 `ROS Twist` 类型的消息，可以发布里程计数据。

特点：

- 可以直接支持 `ping` 声呐和 `Sharp` 红外线传感器；
- 也可以从通用的模拟和数字信号的传感器读取数据；
- 可以控制数字信号的输出；
- 支持 `PWM` 伺服机；
- 如果使用所要求的硬件的话，可以配置基本功能；
- 如果你的 `Arduino` 编程基础好的话，并且具有 `python` 基础的话，你就可以很自由的改动代码来满足你的硬件要求。

注意：

- 官方提供的部分硬件不易采购，需要修改下位机程序，以适配当前硬件。

系统要求：

- 如果只是安装调试下位机，那么不必须安装 `ROS` 系统，只要有 `arduino` 开发环境即可；
- 而上位机调试，适用于 `ROS Indigo` 及更高版本，但是暂不支持最新版本 `noetic`，所以上位机需要使用其它版本的 `ROS`（建议 `melodic`）；

下载：

进入 `ROS` 工作空间的 `src` 目录，输入命令：

```
git clone https://github.com/hbrobotics/ros_arduino_bridge.git
```

1.2. `ros_arduino_bridge` 架构

文件结构说明

```
├── ros_arduino_bridge                                # metapackage (元功能包)
│   ├── CMakeLists.txt
│   └── package.xml
├── ros_arduino_firmware                              # 固件包，更新到 Arduino
│   ├── CMakeLists.txt
│   ├── package.xml
│   └── src
│       └── libraries                                # 库目录
│           └── MegaRobogaiaPololu                  # 针对 Pololu 电机控制器，
```

MegaRobogaia 编码器的头文件定义

			— commands.h	#定义命令头文件
			— diff_controller.h	#差分轮 PID 控制头文件
			— MegaRobogaiaPololu.ino	#PID 实现文件
			— sensors.h	#传感器相关实现, 超声波测距,

Ping 函数

			— servos.h	#伺服器头文件
			— ROSArduinoBridge	#Arduino 相关库定义
			— commands.h	#定义命令
			— diff_controller.h	#差分轮 PID 控制头文件
			— encoder_driver.h	#编码器驱动头文件
			— encoder_driver.ino	#编码器驱动实现, 读取编码器

数据, 重置编码器等

			— motor_driver.h	#电机驱动头文件
			— motor_driver.ino	#电机驱动实现, 初始化控制器,

设置速度

			— ROSArduinoBridge.ino	#核心功能实现, 程序入口
			— sensors.h	#传感器头文件及实现
			— servos.h	#伺服器头文件, 定义插脚, 类
			— servos.ino	#伺服器实现

			— ros_arduino_msgs	#消息定义包
			— CMakeLists.txt	
			— msg	#定义消息
			— AnalogFloat.msg	#定义模拟 IO 浮点消息
			— Analog.msg	#定义模拟 IO 数字消息
			— ArduinoConstants.msg	#定义常量消息
			— Digital.msg	#定义数字 IO 消息
			— SensorState.msg	#定义传感器状态消息
			— package.xml	
			— srv	#定义服务
			— AnalogRead.srv	#模拟 IO 输入
			— AnalogWrite.srv	#模拟 IO 输出
			— DigitalRead.srv	#数字 IO 输入
			— DigitalSetDirection.srv	#数字 IO 设置方向
			— DigitalWrite.srv	#数字 IO 输出
			— ServoRead.srv	#伺服电机输入
			— ServoWrite.srv	#伺服电机输出
			— ros_arduino_python	#ROS 相关的 Python 包, 用于上

位机, 树莓派等开发板或电脑等。

			— CMakeLists.txt	
			— config	#配置目录
			— arduino_params.yaml	#定义相关参数, 端口, rate,

PID, sensors 等默认参数。由 arduino.launch 调用

			— launch	
--	--	--	----------	--

```

|   └─ arduino.launch           #启动文件
|   └─ nodes
|   └─ arduino_node.py         #python 文件,实际处理节点, 由
arduino.launch 调用, 即可单独调用。
|   └─ package.xml
|   └─ setup.py
|   └─ src                     #Python 类包目录
|       └─ ros_arduino_python
|           └─ arduino_driver.py   #Arduino 驱动类
|           └─ arduino_sensors.py #Arduino 传感器类
|           └─ base_controller.py  #基本控制类, 订阅 cmd_vel 话
题, 发布 odom 话题
|       └─ __init__.py           #类包默认空文件

```

上述目录结构虽然复杂, 但是关注的只有两大部分:

- ros_arduino_bridge/ros_arduino_firmware/src/libraries/ROSArduinoBridge
- ros_arduino_bridge/ros_arduino_python/config/arduino_params.yaml

前者是 Arduino 端的固件包实现, 需要修改并上传至 Arduino 电路板;

后者是 ROS 端的一个配置文件, 相关驱动已经封装完毕, 我们只需要修改配置信息即可。

整体而言, 借助于 ros_arduino_bridge 可以大大提高我们的开发效率。

1.3. 案例实现

基于 ros_arduino_bridge 的底盘实现具体步骤如下:

- 了解并修改 Arduino 端程序主入口 ROSArduinoBridge.ino 文件;
- Arduino 端添加编码器驱动;
- Arduino 端添加电机驱动模块;
- Arduino 端实现 PID 调试;

2. Arduino 端入口

ros_arduino_bridge/ros_arduino_firmware/src/libraries/ROSArduinoBridge 下的 RosArduinoBridge.ino 是 Arduino 端程序的主入口,

源文件(添加中文注释)内容如下:

```

/*****
 * ROSArduinoBridge
 *
 * 可以通过一组简单的串口命令来控制差分机器人并接收回传的传感器与里程计
 * 数据, 默认使用的是 Arduino Mega + Pololu 电机驱动模块, 如果使用其他的
 * 编码器或电机驱动需要重写 readEncoder()与 setMotorSpeed()函数
 * A set of simple serial commands to control a differential drive
 * robot and receive back sensor and odometry data. Default
 * configuration assumes use of an Arduino Mega + Pololu motor
 * controller shield + Robogaia Mega Encoder shield. Edit the
 * readEncoder() and setMotorSpeed() wrapper functions if using
 * different motor controller or encoder method.

```

Created for the Pi Robot Project: <http://www.pirobot.org>

and the Home Brew Robotics Club (HBRC): <http://hbrobotics.org>

Authors: Patrick Goebel, James Nugen

Inspired and modeled after the ArbotiX driver by Michael Ferguson

Software License Agreement (BSD License)

Copyright (c) 2012, Patrick Goebel.

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- * Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.

- * Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS

"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES

(INCLUDING,

BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE

- * POSSIBILITY OF SUCH DAMAGE.

*****/

//是否启用基座控制器

//#define USE_BASE // Enable the base controller code

#undef USE_BASE // Disable the base controller code

/* Define the motor controller and encoder library you are using */

//启用基座控制器需要设置的电机驱动以及编码器驱动


```
#ifndef USE_BASE
/* The Pololu VNH5019 dual motor driver shield */
#define POLOLU_VNH5019

/* The Pololu MC33926 dual motor driver shield */
//#define POLOLU_MC33926

/* The RoboGaia encoder shield */
#define ROBOGAIA

/* Encoders directly attached to Arduino board */
//#define ARDUINO_ENC_COUNTER

/* L298 Motor driver*/
//#define L298_MOTOR_DRIVER
#endif

//是否启用舵机
#define USE_SERVOS // Enable use of PWM servos as defined in servos.h
//#undef USE_SERVOS // Disable use of PWM servos

/* Serial port baud rate */
//波特率
#define BAUDRATE 57600

/* Maximum PWM signal */
//最大 PWM 值
#define MAX_PWM 255

//根据 Arduino 型号来包含对应的头文件
#if defined(ARDUINO) && ARDUINO >= 100
#include "Arduino.h"
#else
#include "WProgram.h"
#endif

/* Include definition of serial commands */
//串口命令
#include "commands.h"

/* Sensor functions */
//传感器文件
#include "sensors.h"
```

```
/* Include servo support if required */
//如果启用舵机，需要包含的头文件
#ifdef USE_SERVOS
    #include <Servo.h>
    #include "servos.h"
#endif

```

```

//如果启用基座控制器需要包含的头文件
#ifdef USE_BASE
    /* Motor driver function definitions */
    #include "motor_driver.h" //电机驱动

    /* Encoder driver function definitions */
    #include "encoder_driver.h" //编码器驱动

    /* PID parameters and functions */
    #include "diff_controller.h" //PID 调速

    /* Run the PID loop at 30 times per second */
    #define PID_RATE          30      // Hz 调速频率

    /* Convert the rate into an interval */
    const int PID_INTERVAL = 1000 / PID_RATE; //调速周期

    /* Track the next time we make a PID calculation */
    unsigned long nextPID = PID_INTERVAL;

    /* Stop the robot if it hasn't received a movement command
    in this number of milliseconds */
    #define AUTO_STOP_INTERVAL 2000 //自动结束时间(可按需修改)
    long lastMotorCommand = AUTO_STOP_INTERVAL;
#endif

/* Variable initialization */

// A pair of variables to help parse serial commands (thanks Fergs)
int arg = 0;
int index = 0;

// Variable to hold an input character
char chr;

// Variable to hold the current single-character command
char cmd;
```

```
// Character arrays to hold the first and second arguments
char argv1[16];
char argv2[16];

// The arguments converted to integers
long arg1;
long arg2;

/* Clear the current command parameters */
//重置命令
void resetCommand() {
    cmd = NULL;
    memset(argv1, 0, sizeof(argv1));
    memset(argv2, 0, sizeof(argv2));
    arg1 = 0;
    arg2 = 0;
    arg = 0;
    index = 0;
}

/* Run a command. Commands are defined in commands.h */
//执行串口命令
int runCommand() {
    int i = 0;
    char *p = argv1;
    char *str;
    int pid_args[4];
    arg1 = atoi(argv1);
    arg2 = atoi(argv2);

    switch(cmd) {
        case GET_BAUDRATE:
            Serial.println(BAUDRATE);
            break;
        case ANALOG_READ:
            Serial.println(analogRead(arg1));
            break;
        case DIGITAL_READ:
            Serial.println(digitalRead(arg1));
            break;
        case ANALOG_WRITE:
            analogWrite(arg1, arg2);
            Serial.println("OK");
    }
```

```
        break;
    case DIGITAL_WRITE:
        if (arg2 == 0) digitalWrite(arg1, LOW);
        else if (arg2 == 1) digitalWrite(arg1, HIGH);
        Serial.println("OK");
        break;
    case PIN_MODE:
        if (arg2 == 0) pinMode(arg1, INPUT);
        else if (arg2 == 1) pinMode(arg1, OUTPUT);
        Serial.println("OK");
        break;
    case PING:
        Serial.println(Ping(arg1));
        break;
#ifdef USE_SERVOS
    case SERVO_WRITE:
        servos[arg1].setTargetPosition(arg2);
        Serial.println("OK");
        break;
    case SERVO_READ:
        Serial.println(servos[arg1].getServo().read());
        break;
#endif

#ifdef USE_BASE
    case READ_ENCODERS:
        Serial.print(readEncoder(LEFT));
        Serial.print(" ");
        Serial.println(readEncoder(RIGHT));
        break;
    case RESET_ENCODERS:
        resetEncoders();
        resetPID();
        Serial.println("OK");
        break;
    case MOTOR_SPEEDS: //传入电机控制命令
        /* Reset the auto stop timer */
        lastMotorCommand = millis();
        if (arg1 == 0 && arg2 == 0) {
            setMotorSpeeds(0, 0);
            resetPID();
            moving = 0;
        }
        else moving = 1;
```

```
    leftPID.TargetTicksPerFrame = arg1;
    rightPID.TargetTicksPerFrame = arg2;
    Serial.println("OK");
    break;
case UPDATE_PID:
    while ((str = strtok_r(p, ":", &p)) != '\0') {
        pid_args[i] = atoi(str);
        i++;
    }
    Kp = pid_args[0];
    Kd = pid_args[1];
    Ki = pid_args[2];
    Ko = pid_args[3];
    Serial.println("OK");
    break;
#endif
default:
    Serial.println("Invalid Command");
    break;
}
}

/* Setup function--runs once at startup. */
void setup() {
    Serial.begin(BAUDRATE);

    // Initialize the motor controller if used */
#ifdef USE_BASE
    #ifdef ARDUINO_ENC_COUNTER
        //set as inputs
        DDRD &= ~(1<<LEFT_ENC_PIN_A);
        DDRD &= ~(1<<LEFT_ENC_PIN_B);
        DDRC &= ~(1<<RIGHT_ENC_PIN_A);
        DDRC &= ~(1<<RIGHT_ENC_PIN_B);

        //enable pull up resistors
        PORTD |= (1<<LEFT_ENC_PIN_A);
        PORTD |= (1<<LEFT_ENC_PIN_B);
        PORTC |= (1<<RIGHT_ENC_PIN_A);
        PORTC |= (1<<RIGHT_ENC_PIN_B);

        // tell pin change mask to listen to left encoder pins
        PCMSK2 |= (1 << LEFT_ENC_PIN_A)|(1 << LEFT_ENC_PIN_B);
        // tell pin change mask to listen to right encoder pins
```

```
PCMSK1 |= (1 << RIGHT_ENC_PIN_A)|(1 << RIGHT_ENC_PIN_B);

// enable PCINT1 and PCINT2 interrupt in the general interrupt
mask
PCICR |= (1 << PCIE1) | (1 << PCIE2);
#endif
initMotorController(); //初始化电机控制
resetPID(); //重置 PID
#endif

/* Attach servos if used */
#ifdef USE_SERVOS
  int i;
  for (i = 0; i < N_SERVOS; i++) {
    servos[i].initServo(
      servoPins[i],
      stepDelay[i],
      servoInitPosition[i]);
  }
#endif
}

/* Enter the main loop. Read and parse input from the serial port
and run any valid commands. Run a PID calculation at the target
interval and check for auto-stop conditions.
*/
void loop() {
  //读取串口命令
  while (Serial.available() > 0) {

    // Read the next character
    chr = Serial.read();

    // Terminate a command with a CR
    if (chr == 13) {
      if (arg == 1) argv1[index] = NULL;
      else if (arg == 2) argv2[index] = NULL;
      runCommand();
      resetCommand();
    }
    // Use spaces to delimit parts of the command
    else if (chr == ' ') {
      // Step through the arguments
      if (arg == 0) arg = 1;
```

```
        else if (arg == 1) {
            argv1[index] = NULL;
            arg = 2;
            index = 0;
        }
        continue;
    }
    else {
        if (arg == 0) {
            // The first arg is the single-letter command
            cmd = chr;
        }
        else if (arg == 1) {
            // Subsequent arguments can be more than one character
            argv1[index] = chr;
            index++;
        }
        else if (arg == 2) {
            argv2[index] = chr;
            index++;
        }
    }
}

// If we are using base control, run a PID calculation at the
appropriate intervals
#ifdef USE_BASE
    if (millis() > nextPID) {
        updatePID(); //PID 调速
        nextPID += PID_INTERVAL;
    }

    // Check to see if we have exceeded the auto-stop interval
    if ((millis() - lastMotorCommand) > AUTO_STOP_INTERVAL) {;
        setMotorSpeeds(0, 0);
        moving = 0;
    }
#endif

// Sweep servos
#ifdef USE_SERVOS
    int i;
    for (i = 0; i < N_SERVOS; i++) {
        servos[i].doSweep();
    }
}
```

```
}  
#endif  
}
```

这其中，需要关注的是基座控制器以及串口命令的相关部分，而由于没有使用舵机，所以舵机控制器部分暂不介绍。

2.1. 串口命令

在主程序中，包含了 `commands.h`，该文件中包含了当前程序预定义的串口命令，可以编译程序并上传至 Arduino 电路板，然后打开串口监视器测试(当前程序并未修改，所以并非所有串口可用)：

- w 可以用于控制引脚电平
- x 可以用于模拟输出

以 LED 灯控制为例，通过串口监视器录入命令：

- w 13 0 == LED 灯关闭
- w 13 1 == LED 灯打开
- x 13 50 == LED 灯 PWM 值为 50

2.2. 启用基座控制器

源码默认没有启用基座控制器、启用了舵机，我们需要启用基座控制器，禁用舵机。修改后代码如下：

```
#define USE_BASE      // Enable the base controller code  
//#undef USE_BASE    // Disable the base controller code  
  
/* Define the motor controller and encoder library you are using */  
#ifdef USE_BASE  
    /* The Pololu VNH5019 dual motor driver shield */  
    // #define POLOLU_VNH5019  
  
    /* The Pololu MC33926 dual motor driver shield */  
    // #define POLOLU_MC33926  
  
    /* The RoboGaia encoder shield */  
    // #define ROBOGAIA  
  
    /* Encoders directly attached to Arduino board */  
    // #define ARDUINO_ENC_COUNTER  
  
    /* L298 Motor driver*/  
    // #define L298_MOTOR_DRIVER  
#endif  
  
// #define USE_SERVOS  // Enable use of PWM servos as defined in  
servos.h  
#undef USE_SERVOS    // Disable use of PWM servos
```

注意：我们没有使用官方的电机驱动模块以及编码器，后期需要自定义电机驱动与编码器实现。

3. Arduino 端编码器驱动

测速是整个 PID 闭环控制中的必须环节，我们必须修改代码适配当前 AB 相编码器，虽然需要重写功能，但是测速部分内容已经封装，只需要实现编码器计数即可，大致实现流程如下：

1. ROSArduinoBridge.ino 中需要注释之前的编码器驱动，添加自定义编码器驱动；
2. encoder_driver.h 中设置编码器引脚并声明初始化函数以及中断函数；
3. encoder_driver.ino 中实现编码器计数以及重置函数；
4. ROSArduinoBridge.ino 中 setup 函数调用编码器初始化函数。
5. 测试

3.1. 定义编码器驱动

ROSArduinoBridge.ino 需要添加编码器宏定义，代码如下：

```
#define USE_BASE          // Enable the base controller code
//#undef USE_BASE        // Disable the base controller code

/* Define the motor controller and encoder library you are using */
#ifdef USE_BASE
    /* The Pololu VNH5019 dual motor driver shield */
    // #define POLOLU_VNH5019

    /* The Pololu MC33926 dual motor driver shield */
    // #define POLOLU_MC33926

    /* The RoboGaia encoder shield */
    // #define ROBOGAIA

    /* Encoders directly attached to Arduino board */
    // #define ARDUINO_ENC_COUNTER
    #define ARDUINO_MY_COUNTER

    /* L298 Motor driver */
    // #define L298_MOTOR_DRIVER

    #define L298P_MOTOR_DRIVER
#endif
```

先去除 #define L298P_MOTOR_DRIVER 的注释，否则后续编译会抛出异常。

3.2. 修改 encoder_driver.h 文件

修改后内容如下：

```
/* *****
Encoder driver function definitions - by James Nugen
***** */

#ifdef ARDUINO_ENC_COUNTER
```

```
//below can be changed, but should be PORTD pins;
//otherwise additional changes in the code are required
#define LEFT_ENC_PIN_A PD2 //pin 2
#define LEFT_ENC_PIN_B PD3 //pin 3

//below can be changed, but should be PORTC pins
#define RIGHT_ENC_PIN_A PC4 //pin A4
#define RIGHT_ENC_PIN_B PC5 //pin A5
#elif defined ARDUINO_MY_COUNTER
#define LEFT_A 21
#define LEFT_B 20
#define RIGHT_A 18
#define RIGHT_B 19
void initEncoders();
void leftEncoderEventA();
void leftEncoderEventB();
void rightEncoderEventA();
void rightEncoderEventB();
#endif

long readEncoder(int i);
void resetEncoder(int i);
void resetEncoders();
```

3.3. 修改 encoder_driver.ino 文件

主要添加内容如下:

```
#elif defined ARDUINO_MY_COUNTER
// 功能:实现左右电机的脉冲计数
// 1.定义计数器
volatile long left_count = 0L;
volatile long right_count = 0L;
// 2.初始化
void initEncoders(){
  pinMode(LEFT_A,INPUT); // 21 --- 2
  pinMode(LEFT_B,INPUT); // 20 --- 3
  pinMode(RIGHT_A,INPUT); // 18 --- 5
  pinMode(RIGHT_B,INPUT); // 19 --- 4

  attachInterrupt(2,leftEncoderEventA,CHANGE);
  attachInterrupt(3,leftEncoderEventB,CHANGE);
  attachInterrupt(5,rightEncoderEventA,CHANGE);
  attachInterrupt(4,rightEncoderEventB,CHANGE);
}
// 3.便携中断的回调函数
void leftEncoderEventA(){
```

```
    if(digitalRead(LEFT_A) == HIGH){
        if(digitalRead(LEFT_B) == HIGH){
            left_count++;
        } else {
            left_count--;
        }
    } else {
        if(digitalRead(LEFT_B) == LOW){
            left_count++;
        } else {
            left_count--;
        }
    }
}

void leftEncoderEventB(){
    if(digitalRead(LEFT_B) == HIGH){
        if(digitalRead(LEFT_A) == LOW){
            left_count++;
        } else {
            left_count--;
        }
    } else {
        if(digitalRead(LEFT_A) == HIGH){
            left_count++;
        } else {
            left_count--;
        }
    }
}

void rightEncoderEventA(){
    if(digitalRead(RIGHT_A) == HIGH){
        if(digitalRead(RIGHT_B) == HIGH){
            right_count++;
        } else {
            right_count--;
        }
    } else {
        if(digitalRead(RIGHT_B) == LOW){
            right_count++;
        } else {
            right_count--;
        }
    }
}
```

```
void rightEncoderEventB(){
    if(digitalRead(RIGHT_B) == HIGH){
        if(digitalRead(RIGHT_A) == LOW){
            right_count++;
        } else {
            right_count--;
        }
    } else {
        if(digitalRead(RIGHT_A) == HIGH){
            right_count++;
        } else {
            right_count--;
        }
    }
}

// 4.实现编码器数据读和重置的函数
// i 的取值是 LEFT 或 RIGHT，是左右轮的标记
long readEncoder(int i) {
    if (i == LEFT) return left_count;
    else return right_count;
}

void resetEncoder(int i) {
    if (i == LEFT){
        left_count=0L;
        return;
    } else {
        right_count=0L;
        return;
    }
}
```

3.4. ROSArduinoBridge.ino 实现初始化

setup 添加语句: initEncoders();

```
void setup() {
    Serial.begin(BAUDRATE);

    // Initialize the motor controller if used */
#ifdef USE_BASE
    #ifdef ARDUINO_ENC_COUNTER
        //set as inputs
        DDRD &= ~(1<<LEFT_ENC_PIN_A);
        DDRD &= ~(1<<LEFT_ENC_PIN_B);
        DDRC &= ~(1<<RIGHT_ENC_PIN_A);
        DDRC &= ~(1<<RIGHT_ENC_PIN_B);
```

```
//enable pull up resistors
PORTD |= (1<<LEFT_ENC_PIN_A);
PORTD |= (1<<LEFT_ENC_PIN_B);
PORTC |= (1<<RIGHT_ENC_PIN_A);
PORTC |= (1<<RIGHT_ENC_PIN_B);

// tell pin change mask to listen to left encoder pins
PCMSK2 |= (1 << LEFT_ENC_PIN_A)|(1 << LEFT_ENC_PIN_B);
// tell pin change mask to listen to right encoder pins
PCMSK1 |= (1 << RIGHT_ENC_PIN_A)|(1 << RIGHT_ENC_PIN_B);

// enable PCINT1 and PCINT2 interrupt in the general interrupt
mask
PCICR |= (1 << PCIE1) | (1 << PCIE2);
#elif defined ARDUINO_MY_COUNTER
    initEncoders();
#endif
initMotorController();
resetPID();
#endif

/* Attach servos if used */
#ifdef USE_SERVOS
    int i;
    for (i = 0; i < N_SERVOS; i++) {
        servos[i].initServo(
            servoPins[i],
            stepDelay[i],
            servoInitPosition[i]);
    }
#endif
}
```

3.5. 测试

编译并上传程序，打开串口监视器，然后旋转车轮，在串口监视器中录入 **e** 即可查看左右编码器计数，录入命令 **r** 可以重置计数。

4. Arduino 端电机驱动

自定义电机驱动的实现与上一节的编码器驱动流程类似：

1. ROSArduinoBridge.ino 中需要注释之前的电机驱动，添加自定义电机驱动；
2. motor_driver.h 中设置左右电机引脚；
3. motor_driver.ino 中实现初始化与速度设置函数；
4. 测试

4.1. 定义电机驱动

ROSArduinoBridge.ino 需要添加电机宏定义, 代码如下:

```
#define USE_BASE      // Enable the base controller code
//#undef USE_BASE     // Disable the base controller code

/* Define the motor controller and encoder library you are using */
#ifdef USE_BASE
    /* The Pololu VNH5019 dual motor driver shield */
    //#define POLOLU_VNH5019

    /* The Pololu MC33926 dual motor driver shield */
    //#define POLOLU_MC33926

    /* The RoboGaia encoder shield */
    //#define ROBOGAIA

    /* Encoders directly attached to Arduino board */
    //#define ARDUINO_ENC_COUNTER
    /* 使用自定义的编码器驱动 */
    #define ARDUINO_MY_COUNTER

    /* L298 Motor driver*/
    //#define L298_MOTOR_DRIVER
    //使用自定义的 L298P 电机驱动
    #define L298P_MOTOR_DRIVER
#endif
```

4.2. 修改 motor_driver.h 文件

修改后内容如下:

```
/* *****
Motor driver function definitions - by James Nugen
***** */

#ifdef L298_MOTOR_DRIVER
    #define RIGHT_MOTOR_BACKWARD 5
    #define LEFT_MOTOR_BACKWARD 6
    #define RIGHT_MOTOR_FORWARD 9
    #define LEFT_MOTOR_FORWARD 10
    #define RIGHT_MOTOR_ENABLE 12
    #define LEFT_MOTOR_ENABLE 13
#elif defined L298P_MOTOR_DRIVER
    #define DIRA 4
    #define PWMA 5
    #define DIRB 7
    #define PWMB 6
#endif
```

```
void initMotorController();  
void setMotorSpeed(int i, int spd);  
void setMotorSpeeds(int leftSpeed, int rightSpeed);
```

4.3. 修改 motor_driver.ino 文件

主要添加内容如下:

```
#elif defined L298P_MOTOR_DRIVER  
// 1.初始化  
void initMotorController(){  
    pinMode(DIRA,OUTPUT);  
    pinMode(PWMA,OUTPUT);  
    pinMode(DIRB,OUTPUT);  
    pinMode(PWMB,OUTPUT);  
}  
// 2.设置单电机的转速  
void setMotorSpeed(int i, int spd){  
    unsigned char reverse = 0;  
  
    if (spd < 0)  
    {  
        spd = -spd;  
        reverse = 1;  
    }  
    if (spd > 255)  
        spd = 255;  
  
    if (i == LEFT) {  
        if (reverse == 0) { //正转  
            //为电机输出高电压(要实际测试)  
            digitalWrite(DIRA,HIGH);  
        } else if (reverse == 1) { //反转  
            //为电机输出低电压(要实际测试)  
            digitalWrite(DIRA,LOW);  
        }  
        analogWrite(PWMA,spd);  
    } else /*if (i == RIGHT) //no need for condition*/ {  
        if (reverse == 0) { //正转  
            //为电机输出低电压(要实际测试)  
            digitalWrite(DIRB,LOW);  
        } else if (reverse == 1) { //反转  
            //为电机输出高电压(要实际测试)  
            digitalWrite(DIRB,HIGH);  
        }  
        analogWrite(PWMB,spd);  
    }  
}
```

```
    }  
}  
// 3. 设置两个电机的转速  
void setMotorSpeeds(int leftSpeed, int rightSpeed){  
    setMotorSpeed(LEFT, leftSpeed);  
    setMotorSpeed(RIGHT, rightSpeed);  
}
```

4.4. 测试

编译并上传程序，打开串口监视器，然后输入命令，命令格式为：m num1 num2，num1 和 num2 分别为单位时间内左右电机各自转动的编码器计数，而默认单位时间为 1/30 秒。

举例，假设车轮旋转一圈编码器计数为 3960 (减速比 90，编码器分辨率 11 且采用 4 倍频计数)，当输入命令为 m 200 100 时：

左电机转速为： $200 * 30 * 60 / 3960 = 90.9$ (r/m)

右电机转速为： $100 * 30 * 60 / 3960 = 45.45$ (r/m)

4. Arduino 端电机驱动

自定义电机驱动的